

Báo cáo Seminar: Web Automation Testing (T02)

Môn học: CS423 / CSC15003 — Software Testing · **Seminar:** T02 — Web Automation Testing

Nhóm: 23KTPM1_02 — Đoàn Thành Phát (23127241), Lê Thiên Phú (23127244), Lý Quốc Thanh (23127262), Nguyễn Đình Thái Hưng (23127373)

SUT: EShop — <https://github.com/ttbhanh/eshop-sut>

1. Giới thiệu

1.1. Bối cảnh

Kiểm thử giữ vai trò then chốt trong đảm bảo chất lượng phần mềm, nhưng kiểm thử thủ công (Manual Testing) ngày càng bộc lộ hạn chế trước tốc độ phát hành nhanh và nhu cầu chạy hồi quy (regression) liên tục — dẫn đến sự trỗi dậy của Automation Testing, trong đó Web Automation Testing là dạng phổ biến nhất do phần lớn ứng dụng doanh nghiệp đều có giao diện web; gần đây, AI tiếp tục thay đổi lĩnh vực này qua khả năng sinh test, tự sửa test khi giao diện đổi (self-healing) và các agent tự lập kế hoạch kiểm thử (chi tiết ở mục 2.5).

1.2. Mục tiêu Seminar

Seminar này nhằm đạt được các mục tiêu sau:

- Giới thiệu khái niệm và cơ sở lý thuyết của Web Automation Testing, làm nền tảng để khảo sát công cụ ở các phần sau.
- Khảo sát và so sánh ba công cụ Web Automation Testing truyền thống phổ biến — Playwright, Selenium, Cypress — theo một bộ tiêu chí thống nhất.
- Khảo sát xu hướng AI-Augmented Testing thông qua ba hướng công cụ đại diện: Mabl, Testim AI và GitHub Copilot/Claude/Codex.
- Trình diễn (demo) một quy trình Web Automation Testing hoàn chỉnh trên hệ thống EShop SUT, thể hiện cả tính năng của công cụ truyền thống lẫn tính năng AI-augmented — đúng yêu cầu bắt buộc của đề tài T02.
- Tổ chức hoạt động thực hành "Locator Brawl" cho khán giả, giúp người tham gia tự trải nghiệm sự khác biệt giữa locator viết tay và locator do AI sinh ra.
- Rút ra khuyến nghị sử dụng công cụ phù hợp cho một đội phát triển web thực tế, dựa trên kết quả khảo sát và demo.

1.3. Phạm vi

- Seminar chỉ tập trung vào **Web Automation Testing** — kiểm thử tự động trên trình duyệt.
- Không đề cập Mobile Automation, Desktop Automation hay API/Contract Testing — các mảng này thuộc những đề tài seminar khác trong cùng môn học (T3, T4, T6).
- Hệ thống dùng để demo là **EShop SUT**: <https://github.com/ttbhanh/eshop-sut>, tập trung vào ba luồng nghiệp vụ Login/Lockout (FR-02), Add-to-Cart (FR-07) và Checkout (FR-08) — lý do chọn được trình bày ở mục 2.3 và 5.3.
- Seminar không nhằm kết luận "công cụ nào tốt nhất tuyệt đối", mà giới hạn kết luận trong bối cảnh cụ thể: seminar 45 phút, hoạt động thực hành tối đa 25 phút, và SUT là một ứng dụng e-commerce cỡ vừa.

2. Cơ sở lý thuyết

Nguồn: khái niệm và thuật ngữ nền tảng đối chiếu với ISTQB Glossary (glossary.istqb.org — kho thuật ngữ chuẩn quốc tế cho software testing) và các bài viết kinh điển được liệt kê trong Reading List của brief T02; nguồn kỹ thuật (DOM, Test Pyramid, Page Object, Flaky Test) trích từ MDN Web Docs, Martin Fowler và Google Testing Blog. Danh sách đầy đủ ở mục Tài liệu tham khảo.

2.1. Manual Testing

Khái niệm: Manual Testing là hình thức kiểm thử trong đó tester trực tiếp thực hiện các bước kiểm tra trên phần mềm mà không dùng script hay công cụ tự động hoá — quan sát giao diện, nhập dữ liệu, so sánh kết quả thực tế với kết quả mong đợi bằng kinh nghiệm cá nhân.

Quy trình thực hiện: đọc yêu cầu/test case → chuẩn bị dữ liệu và môi trường → thực hiện thao tác theo kịch bản → quan sát, ghi nhận kết quả → so sánh với kết quả mong đợi → báo cáo lỗi nếu có.

Ưu điểm: linh hoạt, phù hợp kiểm thử khám phá (exploratory testing) và đánh giá trải nghiệm người dùng — những khía cạnh khó định nghĩa oracle rõ ràng cho máy; không cần đầu tư viết script; dễ phát hiện lỗi bất ngờ nằm ngoài kịch bản định sẵn.

Nhược điểm: tốn thời gian/nhân lực khi cần lặp lại nhiều lần (đặc biệt regression); kết quả phụ thuộc kinh nghiệm và sự tập trung của tester nên dễ bỏ sót lỗi; khó mở rộng quy mô để kiểm tra thường xuyên trong CI/CD.

2.2. Automation Testing

Nguồn: ISTQB Glossary (Test Case, Test Suite, Regression Testing); James Bach — "Test Automation Snake Oil" (1999) cho góc nhìn phản biện về giới hạn của automation.

Khái niệm

Automation Testing là việc dùng công cụ, script hoặc framework để tự động thực thi các bước kiểm thử và so sánh kết quả thực tế với kết quả mong đợi, không cần con người thao tác thủ công ở từng bước. Mục tiêu không phải thay thế hoàn toàn tester, mà tự động hoá các kiểm tra có tính lặp lại cao và có kết quả mong đợi rõ ràng.

Quy trình Automation Testing

Xác định test case phù hợp để tự động hoá → chọn công cụ/framework → viết test script (locator, thao tác, assertion) → chạy test → phân tích report/evidence → bảo trì script khi hệ thống thay đổi.

Lợi ích

- Tăng khả năng lặp lại: cùng kịch bản chạy được nhiều lần với cùng điều kiện.
- Rút ngắn feedback loop: lỗi hồi quy được phát hiện nhanh sau mỗi thay đổi.
- Tăng độ bao phủ regression trên các luồng nghiệp vụ quan trọng.
- Tạo bằng chứng (report, screenshot, video, trace, log) khi lỗi xảy ra, hỗ trợ debug.
- Chạy được tự động trong CI/CD trước khi merge/deploy.

Hạn chế

- Chi phí đầu tư ban đầu để viết script và dựng framework.
- Chi phí bảo trì khi giao diện hoặc nghiệp vụ thay đổi.
- Không thay thế được exploratory testing, usability testing hay đánh giá cảm nhận người dùng.
- Vẫn có thể sai nếu test oracle, dữ liệu test hoặc assertion được thiết kế kém.

Vị trí trong SDLC

Automation Testing thường được đặt ở nhiều điểm trong pipeline: chạy nhanh một tập nhỏ test ở pre-commit/pull request để phát hiện lỗi sớm, chạy bộ regression đầy đủ hơn theo lịch (nightly) để không làm chậm vòng lặp phát triển ban ngày, và chạy lại trước khi release (release gate) như một điều kiện bắt buộc để deploy. Phần kỹ thuật CI/CD cụ thể được trình bày ở mục 3.1 (nhóm E) và minh hoạ thực tế ở mục 6 (Demo), không lặp lại ở đây.

So sánh Manual vs Automation Testing & tiêu chí lựa chọn

Tiêu chí	Manual Testing	Automation Testing
Tốc độ thực thi	Chậm, phụ thuộc con người	Nhanh, chạy được ngoài giờ làm việc
Chi phí ban đầu	Thấp, không cần viết script	Cao hơn, cần thời gian dựng script/framework
Chi phí duy trì lâu dài	Tăng tuyến tính theo số lần chạy	Thấp hơn nếu chạy nhiều lần, nhưng tăng khi UI đổi thường xuyên
Độ chính xác/khả năng lặp lại	Phụ thuộc con người, có thể sai lệch giữa các lần	Nhất quán giữa các lần chạy nếu test ổn định
Phù hợp loại test	Exploratory, usability, trường hợp khó định nghĩa oracle	Regression, smoke test, luồng có expected result rõ ràng

- **Nên Automation** khi: flow chạy lặp lại nhiều lần, giá trị nghiệp vụ/rủi ro cao, expected result rõ ràng.
- **Vẫn cần Manual** khi: đánh giá trải nghiệm người dùng, khám phá lỗi ngoài kịch bản, test chỉ chạy một lần hoặc không có giá trị hồi quy.
- **Kết luận:** hai hướng bổ trợ nhau, không thay thế hoàn toàn — một chiến lược kiểm thử tốt kết hợp cả hai theo đúng bối cảnh (xem thêm Test Pyramid ở mục 2.3).

Một số thuật ngữ

Test Suite là thuật ngữ chuẩn ISTQB; Test Script/Test Runner/Assertion là thuật ngữ phổ biến trong công cụ automation (không phải thuật ngữ chính thức ISTQB) — phân biệt rõ để tránh nhầm là chuẩn quốc tế.

- **Test Script:** đoạn code mô tả các bước thực thi và kiểm tra của một test case.
- **Test Suite (ISTQB):** tập hợp các test script/test procedure được thực thi trong một lần chạy test cụ thể.
- **Test Runner:** công cụ thực thi test script và tổng hợp kết quả pass/fail.
- **Assertion:** câu lệnh kiểm tra kết quả thực tế có khớp với kết quả mong đợi hay không.
- **Test Data:** dữ liệu đầu vào được chuẩn bị để phục vụ việc chạy test.
- **Report:** bản tổng hợp kết quả chạy test (pass/fail, log, evidence) sau khi test suite hoàn tất.
- **Regression Testing (ISTQB):** kiểm thử lại phần mềm đã test trước đó sau khi có thay đổi, để đảm bảo không phát sinh lỗi mới hoặc lộ lỗi cũ ở vùng không thay đổi.
- **Flaky Test:** test cho kết quả không nhất quán (khi pass khi fail) dù code ứng dụng không thay đổi — nguyên nhân và cách kiểm soát chi tiết ở mục 2.4.

2.3. Web Automation Testing

Nguồn: mô hình Test Pyramid theo Martin Fowler (martinfowler.com/bliki/TestPyramid.html).

Khái niệm

Web Automation Testing là một nhánh của Automation Testing, thu hẹp phạm vi vào việc tự động hoá kiểm thử trên môi trường trình duyệt: tự động mở trang, thao tác với giao diện, và kiểm tra kết quả hiển thị hoặc hành vi của ứng dụng web.

Đối tượng kiểm thử

Đối tượng chính là giao diện người dùng (UI) và các luồng nghiệp vụ end-to-end chạy qua trình duyệt — khác với unit test (kiểm tra hàm/class riêng lẻ) hay API test (kiểm tra tầng service, không qua giao diện).

Browser Automation

Nguyên lý chung là điều khiển trình duyệt từ bên ngoài thông qua một kênh giao tiếp — ví dụ chuẩn W3C WebDriver protocol (Selenium) hoặc giao thức riêng dựa trên Chrome DevTools Protocol/browser context (Playwright). Công cụ gửi lệnh (mở trang, click, nhập liệu...) và nhận lại trạng thái DOM để tiếp tục thao tác hoặc kiểm tra. Chi tiết cơ chế của từng công cụ được trình bày ở chương 3.

Workflow của Web Automation

Ở mức khái niệm, một luồng Web Automation Testing thường gồm: mở trang → định vị phần tử (locator) → thực hiện thao tác (click, nhập liệu...) → chờ đồng bộ (synchronization) → kiểm tra kết quả (assertion) → ghi nhận bằng chứng (evidence). Chi tiết kỹ thuật từng bước được trình bày ở mục 2.4.

Tiêu chí chọn test case để tự động hoá

Không phải mọi test case đều nên tự động hoá trên web. Nên ưu tiên các flow chạy lặp lại nhiều lần trong regression, có giá trị nghiệp vụ hoặc rủi ro cao, expected result rõ ràng, dữ liệu có thể chuẩn bị/reset được, và UI tương đối ổn định. Ngược lại, chưa nên tự động hoá các test chỉ chạy một lần, mang tính đánh giá cảm tính (ví dụ giao diện đẹp/xấu), expected result chưa rõ, phụ thuộc dịch vụ ngoài khó kiểm soát, hoặc UI đang thay đổi liên tục khiến chi phí bảo trì vượt lợi ích. Tiêu chí này được áp dụng trực tiếp để chọn ba luồng demo — Login/Lockout, Add-to-Cart, Checkout — ở mục 5.3.

Khi nào nên sử dụng

Theo mô hình Test Pyramid của Martin Fowler, số lượng test nên giảm dần từ đáy lên đỉnh: nhiều unit test (nhanh, rẻ) → ít hơn integration/service test → ít nhất là UI/E2E test ở đỉnh — vì test qua UI có 3 nhược điểm cố hữu: **brittle** (dễ gãy hàng loạt khi hệ thống thay đổi), **expensive** (tốn công cụ, license, thời gian viết/chạy), và **slow** (làm chậm pipeline, khó chạy headless). Vì vậy, UI/E2E test nên tập trung vào các luồng nghiệp vụ có giá trị cao thay vì cố phủ mọi chi tiết giao diện — phần logic nhỏ nên được kiểm tra ở tầng unit/API.

Hạn chế

Web Automation Testing thường chậm hơn unit/API test do phải khởi động trình duyệt và chờ render; dễ flaky hơn do phụ thuộc DOM động, mạng và thời gian tải trang (xem bảng rủi ro ở mục 2.4); và có chi phí bảo trì cao khi giao diện thay đổi thường xuyên.

2.4. Kiến thức nền cần nắm

Phần này cung cấp kiến thức kỹ thuật để hiện thực các bước ở mục 2.3.

Nguồn: DOM theo MDN Web Docs (developer.mozilla.org); Test Oracle theo ISTQB Glossary; Page Object Model theo Martin Fowler; Flaky Test theo Google Testing Blog "Flaky Tests at Google and How We Mitigate Them" (2016) — danh sách đầy đủ ở mục Tài liệu tham khảo.

DOM & HTML Element

DOM (Document Object Model) là "một giao diện lập trình cho tài liệu web" (theo định nghĩa của MDN), biểu diễn nội dung trang web dưới dạng cây logic mà trình duyệt dựng lên từ mã HTML — mỗi nhánh của cây kết thúc ở một node, mỗi thẻ HTML (HTML Element) tương ứng với một node mang theo các thuộc tính (attribute) như **id**, **class**, **name**, **data-***. DOM độc lập với ngôn ngữ lập trình cụ thể và không phải một phần của JavaScript — nó là một Web API mà JavaScript

dùng để truy vấn/thay đổi. Automation tool thao tác với trang web bằng cách truy vấn và tương tác với các node trong DOM, nên hiểu cấu trúc DOM là nền tảng để viết locator ổn định.

Locator

Locator là cách xác định một phần tử cụ thể trong DOM để thao tác hoặc kiểm tra, xếp từ kém ổn định đến ổn định hơn:

- **ID, Name, Class:** dựa vào thuộc tính HTML có sẵn — nhanh nhưng dễ đổi theo style/code.
- **CSS Selector:** linh hoạt, dựa vào cấu trúc/thuộc tính DOM — dễ trở nên brittle nếu chọn theo vị trí lồng nhau.
- **XPath:** biểu thức mạnh nhưng thường dài, khó đọc, dễ gãy khi DOM thay đổi.
- **Role/Label/Text/Test-id:** locator hiện đại, gần với cách người dùng/accessibility tree nhận diện UI, ít phụ thuộc cấu trúc DOM — được Playwright khuyến nghị và là hướng ưu tiên trong best practice hiện nay.

Browser Interaction

- Các thao tác cơ bản: Click, Input, Hover, Scroll, Keyboard, Tabs, Windows.
- Các thao tác đặc biệt:
 - **Upload/Download:** cần xử lý file system thật (không chỉ thao tác DOM) — điểm khác biệt giữa các tool.
 - **Drag & Drop:** nổi tiếng khó automation ổn định, dễ flaky do phụ thuộc sự kiện chuột mô phỏng — liên hệ ngược tới mục Flaky Test bên dưới.

Wait Mechanism

Trang web render bất đồng bộ (gọi API, animation, cập nhật DOM), nên test cần chờ đúng điều kiện thay vì chờ theo thời gian cố định — nếu không sẽ gây flaky hoặc false fail.

- **Static Wait:** dừng một khoảng thời gian cố định (**sleep**) — đơn giản nhưng lãng phí thời gian hoặc vẫn có thể chưa đủ.
- **Implicit Wait:** cấu hình một lần, tool tự chờ tối đa X giây trước khi tìm phần tử — dễ dùng nhưng khó tinh chỉnh cho từng trường hợp.
- **Explicit Wait:** chờ một điều kiện cụ thể (phần tử xuất hiện, enabled...) trước khi thao tác — chính xác hơn nhưng phải khai báo thủ công ở từng bước.
- **Fluent Wait:** biến thể của Explicit Wait, cho phép cấu hình tần suất kiểm tra và bỏ qua một số loại exception trong lúc chờ.
- **Auto Wait:** tool tự động chờ phần tử "actionable" (visible, enabled, stable) trước khi thao tác mà không cần khai báo — cách tiếp cận hiện đại (Playwright, Cypress).

Assertion & Test Oracle

Assertion là câu lệnh kiểm tra kết quả thực tế có khớp với kết quả mong đợi hay không, quyết định test pass/fail. Đúng sau assertion là khái niệm **test oracle** — theo ISTQB Glossary, oracle là "nguồn để xác định kết quả mong đợi nhằm so sánh với kết quả thực tế của phần mềm được kiểm thử"; oracle có thể là hệ thống đối chứng, tài liệu đặc tả, hoặc kiến thức chuyên môn của người kiểm thử, nhưng **không nên là chính source code** đang được test. Assertion quá lỏng lẻo gây False Pass, quá chặt dễ gây False Fail (xem bảng rủi ro bên dưới) — cả hai đều là hệ quả của việc chọn/triển khai oracle chưa tốt. Một xu hướng hiện đại là "web-first assertion" — assertion tự động chờ và thử lại đến khi đúng hoặc hết thời gian, giúp giảm phụ thuộc vào wait mechanism khai báo riêng.

Test Isolation

- Khái niệm: mỗi test nên độc lập, không phụ thuộc thứ tự chạy hay trạng thái do test khác để lại.
- Cách đảm bảo: browser context/session riêng cho mỗi test, chuẩn bị dữ liệu trước test, dọn/reset sau test.
- Vi phạm test isolation là một nguyên nhân phổ biến gây Flaky Test (xem bên dưới).

Rủi ro & Kiểm soát trong Web Automation Testing

Việc chọn sai locator, thiếu wait phù hợp hoặc viết assertion kém là nguồn gốc của phần lớn rủi ro khi làm Web Automation Testing. Theo Google Testing Blog, một flaky test là test "không cho kết quả nhất quán dù không có thay đổi nào trong code" — khác với test fail liên tục (vốn là tín hiệu rõ ràng về lỗi thật); Google ghi nhận đây là vấn đề đáng kể trên quy mô lớn và xử lý bằng cách chạy lại (rerun) khi nghi ngờ flaky, cách ly (quarantine) các test có tỷ lệ flaky cao khỏi luồng CI chính, và theo dõi riêng để xử lý dần. Bảng dưới tổng hợp các rủi ro phổ biến nhất trong bối cảnh Web Automation:

Rủi ro	Nguyên nhân	Hậu quả	Cách kiểm soát
Flaky Test	Timing/network, async UI, dữ liệu chia sẻ, vi phạm test isolation, môi trường không ổn định	Mất niềm tin vào test suite	Auto-wait, test isolation, chạy lặp lại, trace/log khi fail
Brittle Locator	Chọn locator theo vị trí DOM (CSS/XPath sâu)	Fail khi UI đổi nhỏ dù nghiệp vụ không đổi	Ưu tiên locator theo role/label/text/test-id
False Pass	Assertion quá lỏng, self-healing chọn nhầm element	Bug thật bị che giấu	Viết oracle bám requirement, luôn xem evidence khi pass đáng ngờ
False Fail	Assertion quá chặt, dữ liệu test không ổn định	Tốn thời gian debug test thay vì bug thật	Kiểm soát fixture, tránh assert chi tiết không quan trọng
Test chậm	Quá nhiều E2E, setup nặng	Pipeline lâu, ít người chạy	Chọn flow đại diện, chạy song song, ưu tiên unit/API cho logic nhỏ
Khó bảo trì	Copy-paste steps, thiếu helper/Page Object	Mỗi thay đổi UI phải sửa nhiều nơi	Tách helper, đặt tên rõ, dùng Page Object Model

- (Được dùng lại ở mục 6.5 - Đo lường Flakiness và 8.2 - Failure Modes)

Thành phần & Test Design

Một test tự động hoàn chỉnh cần đủ các thành phần sau, không chỉ dừng ở "thao tác được":

- **Precondition:** điều kiện cần có trước khi chạy test (ví dụ: tài khoản đã tồn tại, sản phẩm còn hàng).
- **Test Data/Fixture:** dữ liệu đầu vào được kiểm soát, có thể tạo/reset độc lập giữa các lần chạy.
- **Action:** các bước mô phỏng hành vi người dùng.
- **Cleanup/Teardown:** dọn dữ liệu hoặc trả hệ thống về trạng thái ban đầu sau khi test kết thúc.
- **Evidence:** report, screenshot, video, trace, log — bằng chứng để debug khi test fail.

Ở cấp độ tổ chức cao hơn, các bước trên hợp thành một **Test Case** (ISTQB: "*tập hợp giá trị đầu vào, điều kiện tiên quyết, kết quả mong đợi và điều kiện hậu kiểm, được xây dựng cho một mục tiêu hoặc điều kiện test cụ thể*"); nhiều Test Case cùng phục vụ một mục tiêu nghiệp vụ tạo thành **Test Scenario**; nhiều Test Case liên quan được gom vào **Test Suite** để chạy cùng nhau.

Automation Framework

Framework là bộ khung (thư viện, quy ước tổ chức code, công cụ hỗ trợ) giúp viết và chạy test nhất quán, giảm trùng lặp code giữa các test case — quản lý locator, hành động, dữ liệu test, report và cấu hình chạy ở một nơi tập trung thay vì rải rác trong từng test case.

Mô hình phổ biến nhất là **Page Object Model (POM)**, do Martin Fowler mô tả: "một page object bọc một trang HTML, hoặc một phần của trang, bằng một API riêng cho ứng dụng, cho phép thao tác với các phần tử trên trang mà không cần đào sâu vào HTML." Mỗi trang/màn hình của ứng dụng được đại diện bởi một class chứa các locator và hành động liên

quan đến trang đó; test case chỉ gọi lại các hành động này thay vì viết locator trực tiếp — "bằng cách gói gọn logic thao tác UI vào một chỗ duy nhất, bạn có thể sửa nó mà không ảnh hưởng tới các thành phần khác" (Fowler) — khi UI thay đổi, chỉ cần sửa một class thay vì sửa nhiều test.

2.5. AI trong kiểm thử tự động

Nguồn: James Bach — "Test Automation Snake Oil" (1999, theo Reading List brief T02) cho góc nhìn phản biện áp dụng vào AI hype; mabl — "Self-Healing Test Automation for Autonomous QA" cho cơ chế self-healing thực tế.

AI đang được tích hợp vào automation testing theo nhiều cách khác nhau, có thể phân loại theo hai góc nhìn hỗ trợ nhau: theo **chức năng** mà AI thực hiện, và theo **vai trò** của công cụ AI trong quy trình kiểm thử.

Phân loại theo chức năng

- **AI-assisted:** AI hỗ trợ gợi ý trong lúc con người vẫn chủ động viết/chạy test — ví dụ gợi ý locator, tự động hoàn thành code.
- **AI-generated:** AI sinh ra test script hoàn chỉnh từ mô tả yêu cầu hoặc từ việc quan sát thao tác; con người review lại trước khi dùng.
- **AI-healing:** AI tự động phát hiện và sửa test khi fail do thay đổi nhỏ trên giao diện (self-healing locator). Theo mabl, self-healing chỉ khắc phục **một loại lỗi cụ thể** — locator gãy do DOM đổi — chứ không xử lý được flaky do timing, dữ liệu test, hay lệch môi trường, và càng không phải regression thật của ứng dụng.
- **AI-agent:** AI đóng vai trò tác nhân tự chủ, có thể tự lập kế hoạch, sinh test, chạy và sửa test theo một vòng lặp mà ít cần con người can thiệp từng bước.

Phân loại theo vai trò

- **AI coding assistant** (VD: Copilot): gợi ý test draft, locator, assertion, refactor code. Lợi ích: tăng tốc viết test. Rủi ro: locator mong manh, assertion thiếu nghiệp vụ nếu không review.
- **AI-native / self-healing tool** (VD: Mabl, Testim): tự tìm lại element khi locator gãy, hỗ trợ root cause/evidence. Lợi ích: giảm nhiễu do UI đổi nhỏ. Rủi ro: có thể "chữa" sai và tạo False Pass (xem bảng rủi ro ở mục 2.4).
- Thông điệp seminar: AI tăng tốc tạo/bảo trì test, nhưng con người vẫn phải xác định requirement, test oracle và review output — không dùng AI như "hộp đen". Đây cũng chính là tinh thần bài viết kinh điển của James Bach về việc phản biện các tuyên bố quá đà (snake oil) của nhà cung cấp công cụ automation — một cảnh báo vẫn còn nguyên giá trị khi áp dụng cho làn sóng quảng cáo "AI tự động hoá hoàn toàn" hiện nay.

3. Khảo sát Traditional Automation Tools

3.1. Khung tiêu chí đánh giá

Thay vì liệt kê rời rạc, các tiêu chí được gom thành 6 nhóm dùng **thống nhất** làm khung đề mục cho cả 4 công cụ ở mục 3.2-3.5. Nội dung chi tiết được viết một lần trong từng mục khảo sát; mục 3.6 chỉ tổng hợp lại thành bảng, không lặp lại nội dung.

Nhóm	Gồm các tiêu chí	Vì sao quan trọng
A. Cài đặt & Learning Curve	Ease of Learning, Installation, Programming Language, Community, Documentation	Ảnh hưởng tốc độ setup và onboarding trong seminar
B. Kiến trúc & Execution Model	Kiến trúc, cơ chế điều khiển browser, Browser Support	Quyết định độ ổn định và phạm vi trình duyệt hỗ trợ

Nhóm	Gồm các tiêu chí	Vì sao quan trọng
C. Locator & Synchronization	Locator Strategy, Wait Mechanism	Nguồn gốc chính của flaky test và brittle test
D. Test Execution & Evidence	Assertion/Test Oracle, Parallel Testing, Report/Evidence, Performance, Stability	Quyết định khả năng debug và tốc độ pipeline
E. CI/CD & Maintainability	CI/CD integration, Page Object/helper support, Maintenance Cost	Chi phí duy trì test suite lâu dài
F. AI Tooling	AI Support, Agents/Skills/MCP chính thức từ nhà phát triển	Tiêu chí AI-First bắt buộc của seminar

3.2. Khảo sát Playwright

Nguồn: tài liệu chính thức tại playwright.dev — xem danh sách đầy đủ ở mục Tài liệu tham khảo.

Giới thiệu

Playwright là framework kiểm thử tự động mã nguồn mở do Microsoft phát triển, hỗ trợ viết test bằng Node.js/TypeScript/JavaScript, Python, Java hoặc .NET. Playwright điều khiển ba engine trình duyệt: Chromium (dùng để kiểm thử Chrome/Edge), Firefox (bản build khớp với Firefox Stable gần nhất) và WebKit (dựa trên mã nguồn WebKit mới nhất, mô phỏng hành vi Safari).

A. Cài đặt & Learning Curve

Cài đặt chỉ bằng một lệnh — `npm init playwright@latest` (hoặc tương đương với yarn/pnpm) — tự động dựng project mẫu, cài trình duyệt và file cấu hình; yêu cầu Node.js bản 22.x/24.x/26.x trở lên. Playwright có công cụ `codegen`: ghi lại thao tác thực hiện trực tiếp trên trình duyệt và sinh ra code test tương ứng, giúp người mới làm quen nhanh với API mà không cần thuộc cú pháp.

B. Kiến trúc & Execution Model

Playwright duy trì các bản build engine riêng (patched builds) cho Chromium, Firefox, WebKit thay vì phụ thuộc trình duyệt cài sẵn trên máy, giúp hành vi nhất quán giữa các môi trường chạy test. Mỗi test có thể chạy trong một **browser context** riêng — một phiên trình duyệt cô lập (cookie, storage, cache riêng) — cho phép nhiều test chạy song song mà không ảnh hưởng lẫn nhau; đồng thời hỗ trợ giả lập thiết bị (mobile emulation) và chọn kênh trình duyệt cụ thể (Chrome, Edge...).

C. Locator & Synchronization

Playwright khuyến nghị nhóm locator "user-facing" — mô phỏng cách người dùng và công nghệ hỗ trợ (assistive technology) nhận diện trang: `getByRole`, `getByText`, `getByLabel`, `getByPlaceholder`, `getByAltText`, `getByTitle`, `getByTestId` — thay vì CSS/XPath vốn gắn chặt với cấu trúc DOM và dễ gãy khi DOM thay đổi.

Trước khi thực hiện một hành động (ví dụ `click`), Playwright tự động kiểm tra các điều kiện "actionability": phần tử phải **Visible** (có kích thước, không ẩn), **Stable** (đã hết animation), **Enabled**, **Receives Events** (không bị phần tử khác che), và locator phải khớp đúng một phần tử duy nhất — hành động chỉ thực hiện khi mọi điều kiện được thỏa, loại bỏ phần lớn race condition do timing.

D. Test Execution & Evidence

Assertion của Playwright ("web-first assertion", ví dụ `await expect(locator).toBeVisible()`) tự động thử lại — mặc định tối đa 5 giây — cho đến khi điều kiện đúng hoặc hết thời gian, thay vì kiểm tra một lần rồi fail ngay, phù hợp với nội

dung web load bất đồng bộ. Khi test fail, **Trace Viewer** cho phép xem lại từng bước hành động, DOM snapshot có thể tương tác (mở được DevTools), network request và console log; mặc định trace được ghi ở lần retry đầu tiên (**on-first-retry**) để cân bằng giữa lượng thông tin và tài nguyên lưu trữ.

E. CI/CD & Maintainability

Playwright chạy song song theo mặc định bằng nhiều worker process độc lập, mỗi worker giữ trình duyệt riêng; có thể chia nhỏ để chạy trên nhiều máy CI cùng lúc bằng cơ chế sharding (`npx playwright test --shard=2/3`). Playwright cung cấp hướng dẫn CI chính thức cho nhiều nhà cung cấp, kèm cấu hình mẫu sẵn cho GitHub Actions: cài trình duyệt bằng `npx playwright install --with-deps`, chạy bằng `npx playwright test`, sau đó lưu HTML report làm artifact.

F. AI Tooling (chính thức — Playwright Test Agents qua Playwright MCP server)

- Planner: khám phá app, sinh test plan dạng Markdown
- Generator: chuyển test plan thành Playwright TypeScript spec
- Healer: tự động chẩn đoán và sửa test fail (theo Microsoft, >75% thành công với lỗi selector)

Ưu điểm

- Auto-wait (actionability check) + web-first assertion giúp giảm flaky test ngay từ thiết kế, không cần tự viết wait thủ công.
- Trace Viewer ghi lại DOM snapshot/network/console theo từng bước — dễ debug và dễ dùng làm minh chứng khi demo.
- Chạy đa trình duyệt (Chromium/Firefox/WebKit) từ cùng một codebase; parallel hoá theo worker mặc định giúp pipeline nhanh, có sharding cho CI quy mô lớn.
- Có Test Agents (Planner/Generator/Healer) chính thức — hỗ trợ AI-First tốt nhất trong 3 công cụ truyền thống được khảo sát.

Nhược điểm

- Gắn với test runner riêng của Playwright, không linh hoạt ghép nhiều runner/thư viện khác như Selenium.
- Framework tương đối mới hơn Selenium nên tài liệu/cộng đồng bên thứ ba (blog, khoá học, Stack Overflow) chưa nhiều bằng.
- Bắt buộc biết code (TypeScript/JavaScript/Python/Java/.NET), không phù hợp người không rành lập trình như công cụ low-code ở chương 4.

Phù hợp với trường hợp nào

- Team có khả năng code, cần automation nhanh, ổn định, đa trình duyệt, và muốn tận dụng AI tooling chính thức ngay trong framework — phù hợp làm công cụ chính cho demo EShop trong seminar này.

3.3. Khảo sát Selenium

Nguồn: tài liệu chính thức tại selenium.dev — xem danh sách đầy đủ ở mục Tài liệu tham khảo.

Giới thiệu

Selenium là "an umbrella project" — một dự án ô gồm nhiều công cụ/thư viện phục vụ tự động hoá trình duyệt, ra đời từ rất sớm và trở thành chuẩn công nghiệp cho browser automation. Selenium chính thức hỗ trợ 6 ngôn ngữ: Java, Python, C#, Ruby, JavaScript, Kotlin. Dự án gồm 4 thành phần chính: **WebDriver** (lỗi điều khiển trình duyệt), **Selenium IDE** (extension ghi/phát lại thao tác không cần code), **Selenium Grid** (server điều phối chạy song song trên nhiều máy/trình duyệt), và **Selenium Manager** (công cụ dòng lệnh tự động quản lý driver/trình duyệt, đang ở giai đoạn Beta).

A. Cài đặt & Learning Curve

Cài đặt bắt đầu bằng việc thêm thư viện Selenium theo ngôn ngữ đã chọn (Maven/pip/NuGet/npm/gem...). Vì Selenium chỉ là thư viện điều khiển trình duyệt (không đi kèm test runner hay assertion), người dùng cần tự ghép thêm test runner (JUnit/TestNG cho Java, pytest cho Python...) và thư viện assertion — khiến learning curve ban đầu cao hơn Playwright/Cypress vốn có sẵn runner tích hợp.

B. Kiến trúc & Execution Model

WebDriver là một **W3C Recommendation** — giao tiếp giữa code test và trình duyệt tuân theo chuẩn W3C WebDriver protocol đã được chuẩn hoá, không phải giao thức riêng của một hãng. Mỗi trình duyệt cần một driver riêng dịch lệnh WebDriver thành thao tác thực tế (chromedriver cho Chrome, geckodriver cho Firefox...). **Selenium Grid** giải quyết bài toán chạy song song trên nhiều máy và nhiều phiên bản trình duyệt/hệ điều hành khác nhau, bằng cách định tuyến lệnh từ client tới đúng phiên trình duyệt từ xa.

C. Locator & Synchronization

Selenium hỗ trợ 8 chiến lược locator qua class **By**: id, name, className, cssSelector, xpath, linkText, partialLinkText, tagName — cùng với **relative locators** (định vị phần tử dựa trên vị trí tương đối so với phần tử khác) được thêm từ Selenium 4.

Về đồng bộ, Selenium **không có auto-wait/actionability check mặc định** như Playwright hay Cypress — implicit wait mặc định là 0 giây (không chờ). Người viết test phải tự cấu hình: **Implicit Wait** (cấu hình một lần, áp dụng toàn cục cho mọi lần tìm phần tử), **Explicit Wait** (dùng **WebDriverWait** chờ một điều kiện cụ thể, tùy chỉnh được polling interval và exception bỏ qua), hoặc **Fluent Wait** (biến thể tùy biến sâu hơn của Explicit Wait). Tài liệu chính thức cảnh báo không nên trộn lẫn implicit và explicit wait vì có thể gây thời gian chờ khó đoán.

D. Test Execution & Evidence

Selenium không có report/evidence tích hợp sẵn — cần ghép thêm thư viện bên thứ ba (Extent Report, Allure...) để xuất report, chụp screenshot hay quay video; cũng không có công cụ tương đương Trace Viewer của Playwright.

E. CI/CD & Maintainability

Vì không có CLI test runner tích hợp sẵn, việc chạy Selenium trong CI/CD phải tự ghép qua test runner của ngôn ngữ (Maven/Gradle, pytest, npm scripts...), kết hợp Selenium Grid nếu cần chạy song song trên nhiều máy. Về khả năng bảo trì, tài liệu chính thức khuyến nghị mạnh mẽ áp dụng **Page Object Model**: đóng gói locator và thao tác của từng trang vào một class riêng, tách biệt code test khỏi code đặc thù trang — khi UI đổi chỉ cần sửa một class thay vì sửa nhiều test.

F. AI Tooling (chưa chính thức)

Selenium hiện chưa có bộ AI Agents/Skills chính thức do dự án công bố. Trên thị trường chỉ tồn tại các MCP server và SKILL.md do cộng đồng/bên thứ ba xây dựng độc lập (đã khảo sát ở mục 3.7), không được duy trì hay đảm bảo bởi nhóm phát triển Selenium.

Ưu điểm

- Ecosystem lâu đời và rộng nhất trong 3 công cụ — nhiều tài liệu, khoá học, câu trả lời có sẵn trên cộng đồng.
- Hỗ trợ 6 ngôn ngữ chính thức — dễ tích hợp vào codebase đã có sẵn ở ngôn ngữ bất kỳ trong số đó.
- Selenium Grid mạnh cho nhu cầu chạy song song quy mô lớn, nhiều phiên bản trình duyệt/hệ điều hành khác nhau.
- WebDriver protocol là chuẩn W3C, không phụ thuộc một hãng — nền tảng cho nhiều công cụ thương mại khác (Katalon, BrowserStack...).

Nhược điểm

- Không có auto-wait mặc định — dễ viết wait sai (trộn implicit/explicit) dẫn đến flaky test hoặc test chậm không cần thiết.
- Không có report/evidence/trace viewer tích hợp sẵn — phải tự ghép nhiều thư viện, tăng effort setup ban đầu.
- Chưa có AI tooling chính thức, đi sau Playwright và Cypress rõ rệt ở tiêu chí AI-First.

Phù hợp với trường hợp nào

- Team đã có hệ thống automation Selenium từ trước, cần ngôn ngữ ngoài JS/TS/Python (ví dụ Java/C#/Ruby), hoặc cần chạy song song quy mô lớn trên nhiều trình duyệt/hệ điều hành qua Grid — phù hợp làm benchmark lý thuyết hoặc backup trong seminar này hơn là công cụ chính cho demo ngắn.

3.4. Khảo sát Cypress

Nguồn: tài liệu chính thức tại docs.cypress.io — xem danh sách đầy đủ ở mục Tài liệu tham khảo.

Giới thiệu

Cypress là một "quality platform" mã nguồn mở do Cypress.io phát triển và duy trì, tích hợp end-to-end testing, component testing và kiểm tra accessibility trong cùng một công cụ. Cypress chỉ hỗ trợ viết test bằng JavaScript/TypeScript — không đa ngôn ngữ như Selenium hay Playwright.

A. Cài đặt & Learning Curve

Cài đặt qua `npm install cypress --save-dev`, đi kèm **Test Runner** dạng GUI cho phép xem trực tiếp trình duyệt khi test chạy và tua lại từng bước — giúp người mới học nhanh và debug trực quan hơn so với chỉ dùng dòng lệnh.

B. Kiến trúc & Execution Model

Khác biệt kiến trúc lớn nhất của Cypress so với Selenium/Playwright: Cypress chạy **bên trong** trình duyệt, "cùng vòng lặp thực thi (run loop) với ứng dụng đang được test", thông qua một Node.js server process liên tục đồng bộ với trình duyệt. Nhờ chạy trong cùng tiến trình, Cypress có quyền truy cập trực tiếp vào window, DOM, timer, service worker... của ứng dụng — nhưng đổi lại khó "nói chuyện" với thế giới bên ngoài trình duyệt hơn. Ngược lại, Selenium chạy hoàn toàn bên ngoài, gửi lệnh JSON qua HTTP tới driver (chromedriver/geckodriver) rồi mới dịch thành lệnh trình duyệt thực. Kiến trúc trong-trình-duyet giúp Cypress nhanh hơn (không có round-trip mạng) nhưng cũng là nguyên nhân khiến Cypress giới hạn hơn ở kịch bản multi-tab/multi-origin.

C. Locator & Synchronization

Cypress khuyến nghị chính thức: **không** chọn phần tử theo thuộc tính CSS như `id`, `class`, `tag` vì dễ đổi theo style/code; thay vào đó nên thêm thuộc tính `data-*` riêng cho test (ví dụ `data-cy`, `data-test`), tách biệt hoàn toàn khỏi thay đổi giao diện hay hành vi JavaScript.

Về đồng bộ, Cypress phân biệt 3 loại lệnh: **Query** (`.get()`, `.find()`...) tự động thử lại (retry) toàn bộ chuỗi truy vấn; **Assertion** (`.should()`, `.and()`) khi fail sẽ kích hoạt retry lại các query phía trước; **Action** (`.click()`, `.type()`...) tự bản thân không retry, nhưng có sẵn assertion ngầm kiểm tra phần tử actionable (visible, không bị che, không disabled) trước khi thực hiện. Cơ chế "retry-ability" này giúp test không cần sửa gì khi ứng dụng render chậm hơn dự kiến — trong thời gian timeout mặc định 4 giây, Cypress tự thử lại đến khi đúng hoặc hết giờ.

D. Test Execution & Evidence

Cypress tự động chụp screenshot khi test fail lúc chạy bằng `cypress run` (không áp dụng khi mở bằng `cypress open`) — có thể tắt qua cấu hình `screenshotOnRunFailure`. Quay video **không** bật mặc định, cần cấu hình `video: true`; khi dùng cờ `--record`, video được nén và upload lên Cypress Cloud.

E. CI/CD & Maintainability

Chạy CI chỉ cần 2 bước: cài Cypress rồi chạy `npm cypress run`; hỗ trợ chính thức nhiều CI provider (GitHub Actions, CircleCI, GitLab CI, Jenkins, AWS CodeBuild) kèm Docker image dựng sẵn. Điểm cần lưu ý: **chạy song song (parallelization) bắt buộc phải ghi nhận (record) lên Cypress Cloud** — dùng cờ `--record --parallel` để Cypress Cloud tự cân bằng tải các spec file giữa các máy. Về khả năng bảo trì, tài liệu chính thức nhấn mạnh mỗi test phải chạy độc lập được (test isolation) — nên đặt phần khởi tạo dữ liệu ở `beforeEach` thay vì dọn dẹp ở `afterEach`, để đảm bảo trạng thái nhất quán kể cả khi test trước đó bị gián đoạn.

F. AI Tooling (chính thức)

- AI Skills: `cypress-author` (viết/sửa test), `cypress-explain` (review test), `cypress-docs` (tra cứu docs) — phân phối dưới dạng SKILL.md cho Claude Code/Cursor/Copilot
- Cloud MCP: server MCP để AI assistant truy vấn dữ liệu run/flaky test từ Cypress Cloud (GA 20/05/2026)
- Không có agent tự trị kiểu Planner/Generator/Healer — skill chỉ hướng dẫn AI assistant bên ngoài, không tự sinh/sửa test độc lập

Ưu điểm

- Retry-ability tích hợp sẵn cho query/assertion + assertion ngầm kiểm tra actionable cho action — giảm flaky gần như không cần cấu hình thêm.
- Test Runner trực quan, dễ debug bằng mắt; evidence (screenshot khi fail, video khi bật) có sẵn không cần thư viện ngoài.
- Khuyến nghị `data-*` attribute chính thức giúp locator ổn định, tách biệt khỏi thay đổi style.
- Có AI Skills + Cloud MCP chính thức, hỗ trợ tốt việc AI assistant viết/review test đúng convention Cypress.

Nhược điểm

- Chỉ hỗ trợ JavaScript/TypeScript, không đa ngôn ngữ như Selenium/Playwright.
- Kiến trúc chạy trong trình duyệt khiến hạn chế hơn ở kịch bản multi-tab/multi-origin so với Playwright.
- Chạy song song phụ thuộc Cypress Cloud (ngoài mức free tier là dịch vụ trả phí), không tự do như sharding cục bộ của Playwright.
- Không có agent AI tự trị (Planner/Generator/Healer) như Playwright — AI Skills chỉ dừng ở mức hướng dẫn AI assistant bên ngoài.

Phù hợp với trường hợp nào

- Team frontend thuần JavaScript/TypeScript, ưu tiên trải nghiệm debug trực quan và tốc độ phản hồi nhanh trong lúc phát triển — phù hợp làm backup truyền thống tốt cho demo EShop nếu Playwright gặp sự cố cài đặt.

3.5. Khảo sát Puppeteer

Nguồn: tài liệu chính thức tại pptr.dev — xem danh sách đầy đủ ở mục Tài liệu tham khảo.

Giới thiệu

Puppeteer là thư viện JavaScript/TypeScript mã nguồn mở do Google phát triển, cung cấp API mức cao để điều khiển Chrome/Chromium và Firefox qua DevTools Protocol, và WebKit qua WebDriver BiDi. Về kiến trúc, Puppeteer là "người anh em" gần nhất của Playwright — nhiều kỹ sư từng làm Puppeteer tại Google sau đó sang Microsoft xây dựng Playwright, kế thừa nhiều triết lý thiết kế.

A. Cài đặt & Learning Curve

Cài qua `npm install puppeteer`. Chỉ hỗ trợ JavaScript/TypeScript, không đa ngôn ngữ như Playwright/Selenium. Quan trọng nhất: **Puppeteer không đi kèm test runner hay assertion library** — đây là thư viện điều khiển trình duyệt thuần túy, cần tự ghép với Jest/Mocha/Jasmine để viết test thực sự, khác với Playwright/Cypress vốn có sẵn framework test đầy đủ.

B. Kiến trúc & Execution Model

Cùng họ kiến trúc với Playwright: điều khiển trình duyệt qua DevTools Protocol/WebDriver BiDi thay vì WebDriver protocol truyền thống của Selenium. Puppeteer nay đã hỗ trợ đủ 3 engine (Chromium/Firefox/WebKit) — thu hẹp khoảng cách với Playwright ở tiêu chí trình duyệt hỗ trợ so với các phiên bản trước (vốn chỉ tập trung Chrome/Chromium).

C. Locator & Synchronization

Locator chủ yếu dựa trên CSS selector/XPath truyền thống, không có bộ locator "user-facing" phong phú (role/label/text) như Playwright. Về đồng bộ, `waitForSelector()` là cơ chế wait chính: trả về ngay nếu phần tử đã tồn tại tại thời điểm gọi, ngược lại chờ tới khi xuất hiện hoặc hết timeout (mặc định 30 giây). Đây là auto-wait ở mức cơ bản (chờ một điều kiện cụ thể theo yêu cầu), không phải actionability check tự động toàn diện (5 điều kiện) như Playwright — người viết test vẫn phải chủ động gọi đúng hàm wait ở đúng chỗ.

D. Test Execution & Evidence

Có sẵn khả năng chụp screenshot toàn trang và xuất PDF, nhưng **không có Trace Viewer hay hệ thống report tích hợp sẵn** như Playwright — muốn có report/evidence đầy đủ phải tự ghép qua test runner ngoài hoặc thư viện bên thứ ba.

E. CI/CD & Maintainability

Không có cơ chế parallel/sharding chính thức tích hợp sẵn — muốn chạy song song cần thư viện cộng đồng như `puppeteer-cluster` để quản lý pool nhiều instance Chromium. Chạy được trên mọi CI hỗ trợ Node.js, nhưng không có cấu hình mẫu chính thức riêng cho từng CI provider như Playwright/Cypress.

F. AI Tooling

Puppeteer có một MCP server "tham chiếu" (reference implementation) từng được công bố cùng đợt ra mắt ban đầu của Model Context Protocol (khởi xướng bởi Anthropic, nay thuộc Linux Foundation) — cho phép AI agent điều khiển Chrome headless qua Puppeteer. Đây là MCP mang tính tham chiếu/cộng đồng hơn là sản phẩm AI Agent chính thức do Google tự xây dựng và duy trì như một phần của framework — khác với Playwright Test Agents (Planner/Generator/Healer) do chính Microsoft phát triển.

Ưu điểm

- Thư viện nhẹ, linh hoạt, được Google duy trì tích cực, cộng đồng lớn nhờ lịch sử lâu đời.
- Đã hỗ trợ đủ 3 engine trình duyệt như Playwright — thu hẹp khoảng cách về browser support.
- Có MCP server tham chiếu giúp AI agent điều khiển trình duyệt dễ dàng.

Nhược điểm

- Không có test runner/assertion/report/Trace Viewer tích hợp sẵn — phải tự ghép nhiều mảnh (test runner + assertion lib + report + parallel) mới thành một bộ automation hoàn chỉnh, tốn thời gian setup hơn Playwright/Cypress trong phạm vi một seminar ngắn.
- Chỉ hỗ trợ JavaScript/TypeScript.
- Locator vẫn thiên về CSS/XPath, không có bộ locator user-facing hiện đại như Playwright — cùng rủi ro brittle locator như Selenium.
- Không có AI Agent chính thức do Google xây dựng (khác Playwright Test Agents).

Phù hợp với trường hợp nào

- Team đã quen hệ sinh thái Puppeteer hoặc cần một thư viện điều khiển trình duyệt nhẹ để tự ghép vào kiến trúc test riêng (ví dụ kết hợp cùng công cụ scraping/performance testing) — trong seminar này, phù hợp làm ví dụ đối chứng cho thấy vì sao một framework test đầy đủ (Playwright) tiết kiệm thời gian hơn một thư viện điều khiển trình duyệt thuần tuý (Puppeteer) khi mục tiêu là viết test nhanh trong thời gian ngắn.

3.6. Bảng so sánh

Tổng hợp lại từ nội dung đã viết ở mục 3.2-3.5 (không viết nội dung mới ở đây, chỉ rút gọn thành bảng).

Nhóm tiêu chí (theo 3.1)	Playwright	Selenium	Cypress	Puppeteer
A. Cài đặt & Learning Curve	1 lệnh (<code>npm init playwright@latest</code>), có <code>codegen</code> ; 5 ngôn ngữ	Cần tự ghép test runner + assertion lib; learning curve cao nhất; 6 ngôn ngữ	<code>npm install cypress</code> ; Test Runner GUI trực quan; chỉ JS/TS	<code>npm install puppeteer</code> ; không có test runner/assertion sẵn; chỉ JS/TS
B. Kiến trúc & Execution Model	Patched-build engine riêng (Chromium/Firefox/WebKit); browser context cô lập	W3C WebDriver protocol; driver riêng từng browser; điều khiển từ ngoài	Chạy trong trình duyệt, cùng run loop với ứng dụng qua Node.js server process	DevTools Protocol/WebDriver BiDi; cùng họ kiến trúc Playwright, đủ 3 engine
C. Locator & Synchronization	Locator user-facing (role/label/text/test-id) + auto actionability check (5 điều kiện)	8 chiến lược <code>By</code> + relative locators; không auto-wait, phải tự cấu hình Implicit/Explicit/Fluent Wait	Khuyến nghị <code>data-*</code> ; retry-ability cho query/assertion + actionable check ngầm cho action	CSS/XPath truyền thống; <code>waitForSelector()</code> — auto-wait cơ bản, không phải actionability toàn diện
D. Test Execution & Evidence	Web-first assertion tự retry (5s); Trace Viewer (DOM/network/console theo từng bước)	Không có report/evidence tích hợp sẵn; cần Extent Report/Allure bên thứ ba	Screenshot tự động khi fail (<code>cypress run</code>); video tắt mặc định; Test Runner trực quan	Screenshot/PDF built-in; không có report/Trace Viewer tích hợp sẵn
E. CI/CD & Maintainability	Parallel theo worker mặc định + sharding cục bộ miễn phí; CI config mẫu chính thức	Không có CLI runner tích hợp, phải tự ghép pipeline; khuyến nghị POM chính thức	<code>cypress run 2</code> bước; parallel hoá bắt buộc qua Cypress Cloud (trả phí ngoài free tier)	Không có parallel/sharding chính thức; cần <code>puppeteer-cluster</code> cộng đồng
F. AI Tooling	Test Agents (Planner/Generator/Healer) + MCP chính thức	Không có (chỉ MCP/skill cộng đồng)	AI Skills (author/explain/docs) + Cloud MCP chính thức	MCP server tham chiếu (cộng đồng/MCP gốc), không có AI Agent riêng do Google xây dựng

3.7. Hệ sinh thái AI của 4 công cụ trong thời đại AI-First

- So sánh mức độ "agentic" (agent tự trị) vs "guidance" (skill hướng dẫn AI bên ngoài) vs "MCP tham chiếu/cộng đồng" vs "chưa có bộ chính thức".
- Lưu ý về thuật ngữ: **không** gọi Playwright là "AI-Native" — thuật ngữ này trong ngành dành cho các nền tảng SaaS xây từ đầu quanh AI (Mabl, Testim, OctoMind...), tức nhóm đã khảo sát ở Mục 4 (AI-Augmented Direction), để tránh mâu thuẫn với cách phân loại Traditional (Mục 3) vs AI-Augmented (Mục 4) của chính brief T02.
- Kết luận: trong 4 tool truyền thống, Playwright có tích hợp AI chính thức sâu nhất (agent tự trị do chính nhà phát triển xây dựng), Cypress ở mức skill/guidance chính thức, Puppeteer chỉ có MCP tham chiếu/cộng đồng (không phải sản phẩm AI Agent riêng do Google xây), Selenium chưa có bộ chính thức nào — đây là một tiêu chí có lợi cho Playwright khi so trong bối cảnh AI-First của seminar.

3.8. Quyết định lựa chọn

- **Tool chính: Playwright.** Dẫn đầu ở 4/6 nhóm tiêu chí đã khảo sát: A (cài đặt 1 lệnh, có **codegen**), C (locator user-facing + auto actionability check giảm flaky ngay từ thiết kế), D (Trace Viewer là evidence mạnh nhất trong 4 công cụ), F (duy nhất có AI Test Agents chính thức — Planner/Generator/Healer). Ở nhóm E, Playwright ngang Cypress nhưng không phụ thuộc dịch vụ cloud trả phí để chạy song song.
- **Backup Tool: Cypress.** Cũng có retry-ability/actionable check tốt và AI Skills chính thức (nhóm F), Test Runner trực quan dễ dùng khi demo trực tiếp trước lớp — chỉ thua Playwright ở đa ngôn ngữ, khả năng multi-tab/multi-origin, và việc chạy song song phụ thuộc Cypress Cloud.
- **Lý do không chọn Selenium làm chính:** thiếu auto-wait mặc định (nhóm C) khiến dễ viết sai wait dẫn đến flaky ngay trong một seminar ngắn; không có report/trace tích hợp sẵn (nhóm D) nên tốn thêm thời gian setup thư viện thứ ba; chưa có AI tooling chính thức (nhóm F) — không đáp ứng tốt tinh thần AI-First của seminar. Selenium vẫn được giữ lại trong báo cáo làm đối chứng lý thuyết vì ecosystem lâu đời và WebDriver protocol chuẩn W3C — phù hợp giải thích vì sao ngành cần các công cụ hiện đại hơn.
- **Lý do không chọn Puppeteer làm chính:** tuy cùng họ kiến trúc và nay đã hỗ trợ đủ 3 engine như Playwright, Puppeteer chỉ là thư viện điều khiển trình duyệt thuần túy — thiếu test runner/assertion/report/Trace Viewer tích hợp sẵn (nhóm A, D), khiến chi phí setup ban đầu không phù hợp với thời lượng demo ngắn của seminar. Puppeteer vẫn được giữ lại làm đối chứng, minh họa trực tiếp lợi ích của việc chọn một framework test đầy đủ (Playwright) thay vì một thư viện điều khiển trình duyệt cần tự ghép thêm nhiều mảnh.

4. Khảo sát AI-Augmented Testing

Các khái niệm nền tảng (AI-assisted, AI-generated, AI-healing, AI-agent) đã trình bày ở mục 2.5. Phần này khảo sát tiêu chí đánh giá và 4 hướng công cụ AI-Augmented cụ thể.

4.1. Khung tiêu chí đánh giá

8 tiêu chí được gom thành 4 nhóm dùng thống nhất làm khung đề mục cho 4 hướng công cụ ở mục 4.2-4.5. Nội dung chi tiết viết một lần trong từng mục khảo sát; mục 4.6 chỉ tổng hợp lại thành bảng.

Nhóm	Gồm các tiêu chí	Vì sao quan trọng
A. Năng lực AI cốt lõi	Khả năng sinh test, Khả năng sửa test (self-healing)	Giá trị cốt lõi phân biệt hướng AI-Augmented với công cụ truyền thống
B. Tích hợp & vận hành	Browser Support, Integration (IDE/CI)	Quyết định công cụ có ghép được vào quy trình demo EShop hay không
C. Thực tế triển khai	Chi phí/license (free tier, trial, student access), Độ trưởng thành, Tính ổn định	Quyết định nhóm có thực sự dùng được trong phạm vi seminar hay không

Nhóm	Gồm các tiêu chí	Vì sao quan trọng
D. Tin cậy & kiểm soát	Khả năng kiểm chứng/audit output AI	Tiêu chí AI-First bắt buộc — đảm bảo AI không được dùng như "hộp đen" (liên hệ mục 2.5)

4.2. Direction Mabl (14-ngày trial, không có free tier vĩnh viễn)

Nguồn: tài liệu chính thức tại mabl.com/help.mabl.com — xem danh sách đầy đủ ở mục Tài liệu tham khảo.

Giới thiệu

mabl là nền tảng kiểm thử dạng SaaS, tự mô tả là "agentic testing platform", được xây dựng dựa trên AI từ năm 2017. mabl gộp performance, accessibility, API và end-to-end UI testing vào cùng một nền tảng — QA/business user tạo test qua giao diện point-and-click hoặc mô tả bằng ngôn ngữ tự nhiên, trong khi developer vẫn có thể mở rộng test bằng JavaScript/Appium snippet hoặc xây trên nền Playwright mã nguồn mở.

A. Năng lực AI cốt lõi

Auto-heal: khi một phần tử thay đổi đủ nhiều khiến mabl không chắc chắn đã tìm đúng, mabl tìm kết quả khớp tốt nhất với "element model" đã ghi nhận trước đó (thuộc tính, phần tử cha, test-id...) bằng cách so khớp từng phần. Nếu chạy trên cloud và cách chuẩn không tìm được, mabl chuyển sang "advanced auto-heal" dùng generative AI để tìm dựa trên độ tương đồng ngữ nghĩa — nhưng chỉ kích hoạt sau khi test đã chạy thành công tối thiểu 5 lần trong một plan.

Visual testing: so sánh screenshot theo 2 chế độ — Machine Learning-based (học phân biệt thay đổi "mong đợi" và "bất thường" dựa trên baseline) hoặc Run-by-run (so từng pixel với lần chạy trước, mặc định khi chưa bật ML). Visual change được xử lý như **cảnh báo**, không tự làm fail test run.

B. Tích hợp & vận hành

Hỗ trợ đa trình duyệt (Chrome, Firefox...) cho browser test và deployment event. Tích hợp CI/CD qua nhiều hình thức: plugin có sẵn cho Jenkins/Bamboo, "deployment event" tự trigger test ngay khi code deploy, CLI/API để nhúng vào pipeline tùy ý, cùng tích hợp sẵn cho GitHub Actions, GitLab CI, CircleCI, Azure Pipelines, Cloud Build, Octopus Deploy, Bitbucket.

C. Thực tế triển khai

mabl không có gói free tier vĩnh viễn — chỉ có free trial 14 ngày cho toàn bộ nền tảng (khác với gợi ý "Mabl (free tier)" trong brief T02 gốc). Giá chính thức không công khai trên trang pricing (yêu cầu liên hệ báo giá); các ước tính không chính thức từ bên thứ ba cho gói thấp nhất khoảng 499 USD/tháng — con số này **chưa được mabl xác nhận**, chỉ nêu để tham khảo mức độ. Về độ trưởng thành, mabl phát triển liên tục từ 2017 và đã có nhiều case study doanh nghiệp.

D. Tin cậy & kiểm soát

mabl chạy **root cause analysis tự động**, phân loại nguyên nhân fail thành: lỗi ứng dụng thật (regression), nhiễu môi trường, hoặc locator mong manh — trước khi người dùng kịp nhận ra cần điều tra. Mỗi bước test được thu thập dữ liệu chi tiết: DOM snapshot, network activity (HAR file), Chrome performance steptrace — hỗ trợ debug và rút ngắn MTTR (Mean Time To Repair). Khi auto-heal, log hiển thị "Find summary" thể hiện độ tin cậy (confidence) của kết quả khớp; nếu confidence thấp, **test sẽ fail thay vì tự động heal sang một match kém** — đây là cơ chế kiểm soát trực tiếp rủi ro False Pass đã nêu ở mục 2.4. Tuy vậy, visual change chỉ dừng ở mức cảnh báo không chặn fail, nên người dùng vẫn cần chủ động xem lại thay vì tin tưởng tuyệt đối vào kết quả pass.

Ưu điểm

- Auto-heal có cơ chế confidence rõ ràng, từ chối heal khi độ tin cậy thấp thay vì đoán bừa — giảm rủi ro False Pass so với self-healing "hộp đen".
- Root cause analysis tự động phân loại nguyên nhân fail — đáp ứng tốt nhóm D (Tin cậy & kiểm soát).
- Tích hợp CI/CD đa dạng, nhiều nền tảng phổ biến đã hỗ trợ sẵn.
- Không chỉ UI test — gộp performance, accessibility, API testing vào một nền tảng duy nhất.

Nhược điểm

- Không có free tier vĩnh viễn, chỉ trial 14 ngày — khó tái lập lâu dài cho seminar hoặc nhóm không có ngân sách.
- Giá không công khai, phải liên hệ báo giá — khó ước lượng chi phí trước khi thử.
- Advanced auto-heal (GenAI) chỉ kích hoạt sau khi test đã chạy ổn định tối thiểu 5 lần — không hữu ích ngay từ lần đầu viết test mới, đúng tình huống demo ngắn trong seminar.
- Visual change chỉ là cảnh báo không chặn fail — dễ bị bỏ qua nếu người dùng không chủ động kiểm tra.

Phù hợp với trường hợp nào

- Team đã có ngân sách/license, cần nền tảng ít code, tự phục hồi test khi UI đổi nhỏ, và muốn root cause analysis tự động để giảm thời gian debug — trong phạm vi seminar này, phù hợp làm ví dụ đối chứng AI-native/self-healing (minh hoạ qua tài liệu/video) hơn là công cụ chạy live do giới hạn 14 ngày trial.

4.3. Direction Testim AI

Nguồn: tài liệu chính thức tại [testim.io/docs.tricentis.com](https://testim.io/docs/tricentis.com) — xem danh sách đầy đủ ở mục Tài liệu tham khảo.

Giới thiệu

Testim là nền tảng kiểm thử web/mobile AI-powered, hiện thuộc Tricentis (sau khi bị mua lại), nổi bật với công nghệ **Smart Locators**.

A. Năng lực AI cốt lõi

Smart Locators: khi ghi test, thuật toán phân tích hàng trăm thuộc tính liên quan đến phần tử và gán trọng số (weight) để định danh phần tử theo cách "toàn diện" — thay vì dùng một selector tĩnh duy nhất như công cụ truyền thống. Smart Locators "học" qua mỗi lần chạy: nếu một số thuộc tính đổi, locator vẫn dùng các thuộc tính còn lại để tìm đúng phần tử.

Auto Improve: nếu điểm locator (locator score) giảm dưới 70%, Testim tự động cải thiện locator để tăng độ ổn định, và thay thế locator cũ bằng locator mới nếu cải thiện thành công.

B. Tích hợp & vận hành

CLI cài qua npm, tích hợp được với mọi hệ thống CI chạy được shell command; hỗ trợ chính thức Jenkins, GitHub Actions, CircleCI. Hỗ trợ chạy song song, cross-browser trên cloud của Testim hoặc trên Selenium-compatible grid; hỗ trợ test mobile trên nhiều thiết bị ảo.

C. Thực tế triển khai

Testim có gói **Community miễn phí vĩnh viễn** — khi hết 14 ngày trial, tài khoản tự chuyển về Community plan (khác với mabl không có free tier). Giới hạn: 1 Community plan/tổ chức, support chỉ qua self-service docs/Slack community, tính năng "có thể thay đổi" (không cam kết cố định). Với mobile, Community plan gồm chạy local không giới hạn với 1 test song song trên thiết bị kết nối. Giá các gói trả phí không công khai (mô hình sales-led); ước tính bên thứ ba cho gói phổ biến nhất khoảng 450 USD/tháng — **chưa được Testim xác nhận chính thức**. TestOps (nền tảng quản lý vận hành testing ở quy mô tổ chức) là điểm mạnh về độ trưởng thành cho môi trường doanh nghiệp.

D. Tin cậy & kiểm soát

Root cause analysis: tự động chụp screenshot mỗi bước, cho phép so sánh màn hình giữa lần chạy hiện tại và lần chạy thành công gần nhất để tìm điểm khác biệt, kèm console log đã parse và network HAR file.

Điểm cần lưu ý: **Auto Improve của Smart Locators áp dụng hoàn toàn tự động, không cần con người phê duyệt** — hệ thống tự phát hiện điểm locator giảm dưới 70%, tự cải thiện, và tự áp dụng thay thế; người dùng chỉ có thể xem lại thay đổi *sau khi* đã xảy ra (qua Revision History hoặc panel Locators), bước thay đổi được đánh dấu icon "AI" trong khoảng 2 tuần để dễ nhận biết. Đây là khác biệt quan trọng so với mabl (có ngưỡng confidence và **từ chối** heal nếu độ tin cậy thấp thay vì tự áp dụng) — nghĩa là rủi ro False Pass do self-healing "chữa sai" (đã nêu ở mục 2.4) cần được giám sát kỹ hơn khi dùng Testim.

Ưu điểm

- Smart Locators + Auto Improve giảm brittle locator qua thời gian mà gần như không cần thao tác thủ công.
- Có Community free plan vĩnh viễn — dễ tái lập lâu dài hơn mabl trong phạm vi ngân sách hạn chế của seminar.
- Root cause analysis (screenshot theo bước, so sánh lần chạy thành công gần nhất, console log, HAR) hỗ trợ debug nhanh.
- TestOps hỗ trợ quản lý quy mô lớn, phù hợp môi trường doanh nghiệp.

Nhược điểm

- Auto Improve áp dụng tự động không cần phê duyệt con người — rủi ro locator bị "sửa sai" mà không ai chủ động biết ngay, chỉ xem lại được sau qua Revision History.
- Giá gói trả phí không công khai, sales-led, khó ước lượng chi phí trước khi liên hệ.
- Community free plan có giới hạn không công bố rõ ràng ("features are subject to change") — khó cam kết tái lập ổn định lâu dài cho activity trên lớp.

Phù hợp với trường hợp nào

- Team cần nền tảng AI-native có free tier vĩnh viễn để thử nghiệm dài hạn, ưu tiên độ ổn định locator theo thời gian hơn kiểm soát chặt từng lần auto-heal — trong seminar này, phù hợp làm ví dụ đối chứng thứ hai bên cạnh mabl để so sánh 2 triết lý self-healing khác nhau (gated theo confidence vs tự động áp dụng).

4.4. Direction Katalon Studio

Nguồn: tài liệu chính thức tại katalon.com/docs.katalon.com — xem danh sách đầy đủ ở mục Tài liệu tham khảo.

Giới thiệu

Katalon Studio là nền tảng kiểm thử xây dựng trên nền Selenium/Appium, bổ sung giao diện đơn giản hoá, quản lý test và report tích hợp sẵn. Ngày 07/04/2026, Katalon ra mắt **"True Platform"** — hợp nhất nhiều AI agent phủ toàn bộ vòng đời chất lượng phần mềm.

A. Năng lực AI cốt lõi

True Platform mô tả các năng lực AI: sinh/cập nhật/tự sửa test liên tục; tự phát hiện thay đổi ứng dụng và thích ứng theo thời gian thực (self-healing); tự phát hiện bug; phân tích rủi ro/xu hướng chất lượng; phân tích yêu cầu; và một "Katalon AI Assistant" nhận lệnh bằng ngôn ngữ tự nhiên, chuyển ý định người dùng thành quy trình end-to-end.

B. Tích hợp & vận hành

Hỗ trợ Chrome, Firefox qua Docker image dựng sẵn (kèm Katalon Studio cài đặt sẵn). Tích hợp CI/CD qua Katalon Plugin cho Jenkins, Docker image, chế độ console cho các CI khác (TeamCity...); chạy trên TestCloud (dịch vụ cloud riêng của Katalon) không cần license Katalon Runtime Engine.

C. Thực tế triển khai

Katalon Studio (IDE desktop) có bản **"Forever Free"** thật sự — miễn phí vĩnh viễn để tạo/chạy test cục bộ. Tuy nhiên, **Katalon Runtime Engine (KRE)** — thành phần bắt buộc để chạy CI/CD, headless, hoặc song song — là license trả phí theo node (ước tính bên thứ ba khoảng 135 USD/user/tháng cho Runtime Engine, 170 USD/user/tháng cho Studio Enterprise, **chưa được Katalon xác nhận chính thức**). Nghĩa là: **miễn phí khi chạy tay trên máy cá nhân, nhưng trả phí khi cần chạy tự động trong CI/CD** — điểm dễ gây hiểu nhầm nếu chỉ đọc nhãn "Forever Free".

D. Tin cậy & kiểm soát

Katalon quảng bá True Platform là "lớp trách nhiệm giải trình" (accountability layer), với thông điệp "mọi hành động do AI thực hiện đều có thể kiểm toán (auditable), giải thích được" và "agent thực thi trong khi con người xác minh và phê duyệt". Tuy nhiên, cùng tài liệu công bố cũng ghi rõ riêng tính năng self-healing hoạt động **"không cần con người can thiệp"** — nghĩa là thông điệp "con người luôn phê duyệt" không áp dụng đồng nhất cho mọi tính năng AI trên platform. Cần đọc kỹ để tránh hiểu nhầm mức độ kiểm soát thực tế, đúng tinh thần phản biện marketing tool automation của James Bach đã nêu ở mục 2.5.

Ưu điểm

- IDE miễn phí vĩnh viễn, đường cong học tập thấp nhờ giao diện đơn giản hoá trên nền Selenium/Appium.
- True Platform phủ rộng nhất trong các hướng AI-Augmented đã khảo sát (test + bug + risk + requirement + trợ lý ngôn ngữ tự nhiên), không chỉ dừng ở self-healing locator.
- Hỗ trợ đa nền tảng thật sự (Web/Mobile/API/Desktop) trong cùng một sản phẩm.

Nhược điểm

- Free chỉ áp dụng cho IDE chạy cục bộ; muốn chạy CI/CD/song song bắt buộc phải mua Katalon Runtime Engine — khó tái lập miễn phí lâu dài cho seminar hoặc nhóm không ngân sách.
- Self-healing hoạt động tự động không cần phê duyệt con người, dù platform tự quảng bá có "governance" — rủi ro False Pass tương tự Testim (mục 4.3), cần xem evidence kỹ thay vì tin tưởng nhãn "auditable".
- True Platform mới ra mắt 04/2026 — độ trưởng thành thực tế (số case study, review dài hạn) chưa nhiều bằng Mabl/Testim vốn đã vận hành nhiều năm.

Phù hợp với trường hợp nào

- Tổ chức đã dùng Selenium/Appium và muốn nâng cấp dần lên giao diện quản lý + AI mà không đổi hoàn toàn hệ sinh thái, hoặc cần kiểm thử đa nền tảng (Web/Mobile/API) trong một công cụ — trong seminar này, phù hợp làm đối chứng AI-native thứ 3, đại diện hướng "low-code xây trên nền code-first cũ", khác với Mabl/Testim (SaaS đóng hoàn toàn) và Copilot/Claude/Codex (thuần coding assistant).

4.5. Direction GitHub Copilot / Claude / Codex

Nguồn: tài liệu chính thức tại docs.github.com, playwright.dev/mcp, và các trang pricing chính thức — xem danh sách đầy đủ ở mục Tài liệu tham khảo.

Giới thiệu

Đây là nhóm 3 AI coding assistant phổ biến — GitHub Copilot (Microsoft/GitHub), Claude Code (Anthropic), OpenAI Codex — đại diện cho hướng "AI hỗ trợ sinh test bằng ngôn ngữ tự nhiên" thay vì một nền tảng test chuyên biệt như Mabl/Testim/Katalon. Cả 3 **không phải test runner**: cần một framework automation thật (ở đây là Playwright, theo lựa chọn tại mục 3.8) đứng phía sau để thực thi test trên trình duyệt.

A. Năng lực AI cốt lõi

Cả 3 công cụ đều sinh test từ mô tả ngôn ngữ tự nhiên hoặc từ code có sẵn — ví dụ GitHub Copilot có lệnh `/tests` sinh bộ test đầy đủ từ một file code. Cần phân biệt rõ 2 chế độ:

- **Chế độ gợi ý/chat** (Copilot Chat cơ bản): chỉ sinh code test, **không tự chạy test** — tài liệu chính thức GitHub ghi rõ người dùng phải tự chạy test bằng công cụ dòng lệnh của framework sau khi nhận code gợi ý.
- **Chế độ agent tự trị** (GitHub Copilot coding agent, Claude Code, OpenAI Codex agent): có thể tự đọc repo, sửa code, **chạy test/linter trong sandbox riêng** (GitHub Actions ephemeral environment với Copilot cloud agent; sandbox riêng với Codex), tự phát hiện test fail và thử sửa lại, rồi mở Pull Request.
- Để điều khiển được trình duyệt thật (không chỉ sinh code tĩnh), các agent này cần kết nối qua **Playwright MCP server** (chuẩn Model Context Protocol chính thức của Microsoft) — cho phép AI agent thao tác trực tiếp trên trình duyệt qua accessibility snapshot, dùng chung được cho VS Code, Claude Code, Cursor...

B. Tích hợp & vận hành

GitHub Copilot tích hợp sâu VS Code và JetBrains, "Agent Mode" đã GA (general availability) trên cả hai IDE từ tháng 3/2026. Claude Code chạy trong terminal, kết nối MCP để điều khiển trình duyệt hoặc công cụ khác. OpenAI Codex chạy như một agent kỹ sư phần mềm tự trị, đọc toàn bộ repo, sửa code nhiều file, chạy test trong sandbox riêng, tự mở PR.

C. Thực tế triển khai

- **GitHub Copilot**: Free (giới hạn, chỉ auto model selection) và **Student (miễn phí cho sinh viên đã xác minh)**, Pro 10 USD/tháng, Pro+ 39 USD/tháng, Max 100 USD/tháng (cá nhân); Business 19 USD/seat/tháng, Enterprise 39 USD/seat/tháng (tổ chức).
- **Claude Code**: **không có gói miễn phí riêng** — tính năng Claude Code chỉ mở khi có gói Pro (17 USD/tháng thanh toán năm, hoặc 20 USD/tháng thanh toán theo tháng) trở lên, hoặc trả theo token qua API; gói Max từ 100 USD/tháng cho nhu cầu cao hơn.
- **OpenAI Codex**: **có Free** (0 USD/tháng, dùng cho tác vụ coding cơ bản), Go 8 USD/tháng, Plus 20 USD/tháng, Pro từ 100 USD/tháng (gấp 5x/20x rate limit so với Plus) (cá nhân); Business 20-25 USD/user/tháng (tổ chức).
- Cả 3 đều đã có chế độ agent tự trị ở trạng thái GA trong năm 2026, cho thấy tính năng đã ổn định hơn giai đoạn preview trước đó.
- Với riêng seminar này, nhóm đã sẵn có: Copilot Student (miễn phí, đã xác minh sinh viên), Claude Code gói Max, và Codex gói Pro — nên hạn chế "không có free tier" của Claude Code không phải rào cản thực tế cho nhóm, dù vẫn là điểm cần lưu ý khi nhóm khác muốn tái lập mà không có sẵn ngân sách. Riêng gói Copilot Student cũng là minh chứng cụ thể cho việc tuân thủ đúng AI Policy của seminar (chỉ dùng free tier/student trial, Department không cấp tài khoản trả phí).

D. Tin cậy & kiểm soát

Tài liệu chính thức của GitHub Copilot khuyến nghị rõ: "luôn phải review code do AI sinh ra, thêm test còn thiếu nếu cần" — không tự động tin tưởng output. Ở chế độ agent tự trị, cả 3 công cụ đều có vòng lặp tự kiểm chứng: chạy test, nếu fail thì tự sửa và chạy lại — nhưng đây là **AI tự kiểm tra AI**, không thay thế việc con người review logic nghiệp vụ, đúng nguyên tắc đã nêu ở mục 2.5 ("AI tăng tốc nhưng con người vẫn phải xác định test oracle"). Vì các agent này có quyền tự sửa code/chạy lệnh, cần giới hạn phạm vi sử dụng (chỉ dùng trên repo demo, không đưa secret/thông tin nhạy cảm vào prompt) và luôn review diff/PR trước khi merge.

Ưu điểm

- Tích hợp trực tiếp vào quy trình code sẵn có (IDE/terminal), không cần học nền tảng SaaS riêng như Mabl/Testim.
- Chế độ agent tự trị có vòng lặp tự chạy - tự sửa - tự chạy lại, giảm nhiều công sức thủ công so với chỉ gợi ý code.
- Có phương án miễn phí/giá thấp phù hợp sinh viên (Copilot Free/Student, Codex Free) — đúng điều kiện tài khoản của brief seminar.
- Kết hợp Playwright MCP cho phép AI agent thao tác trực tiếp trên trình duyệt thật, không chỉ sinh code tĩnh.

Nhược điểm

- Chế độ chat/gợi ý cơ bản (không phải agent) hoàn toàn không tự chạy test — dễ nhầm tưởng AI đã "kiểm chứng" trong khi chưa chạy gì cả.
- Claude Code không có gói miễn phí riêng, tối thiểu 17-20 USD/tháng (gói Pro) — cần cân nhắc ngân sách nếu nhóm khác muốn tái lập mà chưa có sẵn tài khoản trả phí như nhóm này.
- Agent tự trị có quyền sửa code/chạy lệnh — cần giới hạn phạm vi và review kỹ trước khi merge để tránh rủi ro thay đổi ngoài ý muốn.
- Không phải nền tảng test chuyên biệt — không có Trace Viewer, root cause analysis hay TestOps như Mabl/Testim/Katalon; phụ thuộc hoàn toàn vào framework nền (Playwright) để có evidence.
- Dùng cả 3 công cụ song song có thể khiến kết quả demo khó so sánh nếu không chuẩn hoá cùng một prompt/scenario cho cả 3 (xem cách kiểm soát ở mục 4.7).

Phù hợp với trường hợp nào

- Team đã chọn Playwright làm nền tảng chính (mục 3.8) và mỗi thành viên đã quen dùng một AI coding assistant khác nhau với tài khoản sẵn có (Copilot Student, Claude Code Max, Codex Pro) — phù hợp khai thác cả 3 công cụ song song thay vì ép dùng chung một công cụ, miễn là chuẩn hoá cùng scenario để so sánh công bằng, đúng vai trò "human writes scenario → AI drafts test → human audits → run test" mà seminar muốn minh hoạ.

4.6. Bảng so sánh

Tổng hợp lại từ nội dung đã viết ở mục 4.2-4.5 (không viết nội dung mới ở đây, chỉ rút gọn thành bảng).

Nhóm tiêu chí (theo 4.1)	Mabl	Testim AI	Katalon Studio	GitHub Copilot/Claude/Codex
A. Năng lực AI cốt lõi	Record thao tác + auto-heal 2 tầng (chuẩn/GenAI sau ≥5 lần chạy) + visual testing (ML/pixel)	Smart Locators (trọng số nhiều thuộc tính) + Auto Improve tự động khi score <70%	True Platform: sinh/tự sửa test, phát hiện bug, phân tích rủi ro/yêu cầu, trợ lý ngôn ngữ tự nhiên	Sinh test từ ngôn ngữ tự nhiên; agent tự trị chạy/sửa test trong sandbox; cần Playwright MCP để điều khiển trình duyệt thật
B. Tích hợp & vận hành	Đa trình duyệt; nhiều hình thức CI/CD (plugin, deployment event, CLI/API)	CLI qua npm; CI phổ biến (Jenkins/GH Actions/CircleCI); cross-browser qua cloud/Selenium grid	Chrome/Firefox qua Docker image; Jenkins plugin, TestCloud, chế độ console	Tích hợp IDE/terminal (VS Code, JetBrains, CLI); không tự có browser support, phụ thuộc framework nền
C. Thực tế triển khai	Không có free tier, chỉ trial 14 ngày; giá không công khai	Có Community free plan vĩnh viễn (giới hạn không rõ ràng); giá trả phí không công khai	IDE Forever Free, nhưng Runtime Engine (bắt buộc cho CI/CD) trả phí theo node	Copilot có Free/Student; Codex có Free; Claude Code không có free, tối thiểu 17-20 USD/tháng
D. Tin cậy & kiểm soát	Root cause analysis tự động + confidence score, từ chối heal nếu tin cậy thấp	Root cause analysis (screenshot/log/HAR); Auto Improve áp dụng tự động, không cần phê duyệt con người	Tự quảng bá "auditable + con người phê duyệt", nhưng self-healing lại nói rõ hoạt động không cần con người can thiệp —	Cần con người review code/test; agent có vòng lặp tự kiểm nhưng không thay thế review nghiệp vụ

Nhóm tiêu chí (theo 4.1)				
	Mabl	Testim AI	Katalon Studio	GitHub
				Copilot/Claude/Codex
			thông điệp chưa nhất quán	

4.7. Quyết định lựa chọn

- **Direction chính: dùng cả 3 — GitHub Copilot, Claude Code, OpenAI Codex.** Khác với chương 3 (nơi cần một stack demo thống nhất nên chỉ chọn 1 tool chính + 1 backup), ở đây nhóm giữ cả 3 vì mỗi thành viên đã quen dùng một AI coding assistant khác nhau trong công việc hằng ngày, và nhóm đã sẵn có tài khoản cho cả 3 (Copilot Student, Claude Code Max, Codex Pro) — không bị ràng buộc bởi giới hạn free tier như giả định ban đầu ở nhóm C.
- **Cách kiểm soát để so sánh công bằng:** cả 3 công cụ nhận cùng một prompt/scenario mô tả cho cùng một flow (ví dụ Add-to-Cart), sau đó so sánh locator/assertion do từng công cụ sinh ra — vừa đúng tinh thần hoạt động "Locator Brawl" ở mục 7, vừa cho thấy khác biệt thực tế giữa các AI coding assistant thay vì chỉ nói lý thuyết. Không chuẩn hoá prompt sẽ khiến kết quả không so sánh được (đã nêu ở Nhược điểm mục 4.5).
- **Không cần "Backup Direction" riêng** trong trường hợp này: vì không bị giới hạn bởi chi phí/khả dụng của một tài khoản duy nhất, nếu một công cụ gặp sự cố khi demo, các thành viên khác vẫn có công cụ AI riêng để tiếp tục — bản thân việc dùng cả 3 đã đóng vai trò dự phòng lẫn nhau.
- **Lý do không chọn Mabl/Testim/Katalon làm hướng chạy demo:** cả ba đều là nền tảng đóng (SaaS hoặc cần Runtime Engine trả phí), không ghép trực tiếp vào codebase Playwright đã chọn được — muốn demo phải học giao diện riêng và phụ thuộc cloud/license (Mabl không có free tier; Testim có Community free nhưng giới hạn không công bố rõ; Katalon free chỉ áp dụng cho IDE cục bộ, CI/CD vẫn trả phí). Cả ba vẫn được giữ lại trong báo cáo làm **đối chứng AI-native/self-healing** để minh hoạ ba triết lý self-healing khác nhau (Mabl: gated theo confidence; Testim: tự động áp dụng không cần con người duyệt; Katalon: thông điệp governance chưa nhất quán với thực tế self-healing tự động) — nội dung quan trọng cho phần rủi ro AI ở mục 2.4 và 8.2, dù không dùng để chạy demo trực tiếp.
- **Lưu ý AI Policy:** dù đã có tài khoản trả phí, mọi output từ cả 3 công cụ dùng trong demo vẫn phải được review và ghi lại prompt/kết quả trước khi trình bày (theo mục 10 — Khai báo sử dụng AI); mỗi thành viên tự khai báo công cụ mình dùng trong AI Disclosure ([AI-03]).

5. Giới thiệu hệ thống sẽ Demo

5.1. Giới thiệu e-shop SUT

Mục đích

EShop SUT (<https://github.com/ttbhanh/eshop-sut>) là hệ thống thương mại điện tử do giảng viên cung cấp làm SUT (System Under Test) chung cho seminar T02, dùng để nhóm khảo sát và demo Web Automation Testing trên một ứng dụng thật thay vì ví dụ minh hoạ đơn giản.

Kiến trúc

Hệ thống gồm 4 thành phần độc lập:

- **Backend API** — cung cấp REST API cho toàn bộ nghiệp vụ.
- **Frontend Web** — giao diện mua sắm dành cho người dùng cuối; đây là đối tượng demo chính của seminar T02.

- **Admin Panel** — giao diện quản trị dành riêng cho admin.
- **Mobile App** — ứng dụng di động, ngoài phạm vi seminar (theo mục 1.3, seminar chỉ tập trung Web).

Công nghệ

- Backend API: Node.js + Express + SQLite (cổng 3000).
- Frontend Web: React + Vite + Tailwind CSS (cổng 5173).
- Admin Panel: React + Vite + Tailwind CSS (cổng 5174).
- Mobile App: React Native + Expo.

5.2. Các chức năng chính (theo Functional Requirements của SUT)

Account & Authentication

- FR-01: Đăng ký tài khoản, yêu cầu mật khẩu mạnh (≥ 8 ký tự, có chữ hoa/chữ thường/số/ký tự đặc biệt).
- FR-02: Đăng nhập; khóa tài khoản sau 3 lần nhập sai trong 30 giây.
- FR-03: Quên mật khẩu 2 bước qua mã OTP 6 số.
- FR-04: Quản lý hồ sơ cá nhân (tên, số điện thoại, địa chỉ giao hàng).

Shopping

- FR-05: Tìm kiếm và duyệt sản phẩm theo tên.
- FR-06: Xem chi tiết sản phẩm, chọn số lượng.
- FR-07: Giỏ hàng — thêm/xóa/đổi số lượng sản phẩm.
- FR-08: Checkout — chỉ dành cho user đã đăng nhập, tổng tiền được tính ở phía server.
- FR-09: Hệ thống coupon với 5 điều kiện kiểm tra (giá trị đơn hàng tối thiểu, giới hạn lượt dùng...).

Order Management

- FR-10: Trạng thái đơn hàng theo state machine 5 trạng thái (pending $\rightarrow$ confirmed $\rightarrow$ shipping $\rightarrow$ delivered, hoặc canceled ở các bước trung gian).
- FR-11: Xem lịch sử đơn hàng của user.

Admin

- FR-12: Phân quyền theo vai trò (yêu cầu role admin).
- FR-13: Dashboard hiển thị doanh thu và số lượng đơn hàng.
- FR-14 $\rightarrow$ FR-17: CRUD danh mục, sản phẩm, import CSV, coupon.
- FR-18: Quản lý đơn hàng đầy đủ, chuyển trạng thái.
- FR-19: Quản lý người dùng (xem, xóa — trừ tài khoản của chính mình).

FR-20 (mobile) nằm ngoài phạm vi seminar này, theo mục 1.3.

5.3. Các luồng nghiệp vụ được chọn để Demo

Nhóm chọn **3 luồng**: Login + Lockout, Add-to-Cart, và Checkout. Ba luồng này vừa là yêu cầu của brief T02 (mục Study Milestones), vừa thỏa toàn bộ 5 tiêu chí chọn test case để tự động hoá đã nêu ở mục 2.3: chạy lặp nhiều lần trong regression, giá trị nghiệp vụ/rủi ro cao, expected result rõ ràng (có SRS đối chiếu), dữ liệu chuẩn bị và reset được (qua API `/api/register` + `node database.js`), và UI đủ ổn định trong phạm vi seminar.

Flow	Functional Requirement	Lý do chọn
Login + Lockout	FR-02 (kèm FR-22 về form)	Cổng vào của mọi luồng khác — hỏng là chặn toàn hệ thống. Nhiều nhánh với expected result rõ ràng: đăng nhập đúng, sai mật khẩu, và khoá tài khoản sau 3 lần sai. Nhánh lockout là test có trạng thái (stateful): phải reset dữ liệu giữa các lần chạy, nên minh hoạ trực tiếp nguyên tắc Test Isolation ở mục 2.4. Quan trọng nhất: UI nuốt mọi lỗi thành một câu chung chung ("Đăng nhập thất bại"), không phân biệt được <i>sai mật khẩu</i> với <i>bị khoá</i> — buộc phải đặt oracle ở tầng API, minh hoạ đúng luận điểm Test Pyramid ở mục 2.3.
Add-to-Cart	FR-06, FR-07 (kèm FR-23 badge, FR-24 feedback)	Luồng có DOM động nhất: giỏ hàng cập nhật bất đồng bộ sau click, state nằm trong React context. Đây là môi trường lý tưởng để so sánh chiến lược locator và đo flakiness — nên brief cũng chọn chính luồng này cho hoạt động "Locator Brawl" (mục 7). Có nhiều biến thể để đối chiếu với SRS: thêm từ trang chủ (1 click) vs thêm từ trang chi tiết, gộp số lượng khi thêm trùng sản phẩm, badge số lượng trên navbar.
Checkout	FR-08, FR-09 (coupon)	Luồng E2E dài và có giá trị nghiệp vụ/rủi ro cao nhất — liên quan trực tiếp tới tiền. Đi qua nhiều bước (đăng nhập → giỏ hàng → thanh toán → coupon), phụ thuộc trạng thái tích lũy từ 2 luồng trước, nên là đại diện tốt nhất cho E2E thật. Đây cũng là nơi ranh giới client-server lộ rõ: FR-08 yêu cầu backend tự tính lại tổng tiền, nên oracle bắt buộc phải đặt ở tầng dữ liệu chứ không phải ở màn hình — một bài học không luồng nào khác dạy được.

Lý do không chọn các luồng còn lại. FR-03 (quên mật khẩu qua OTP) phụ thuộc mã OTP sinh ngẫu nhiên — expected result không tiền định, chi phí tự động hoá cao so với giá trị demo. FR-13→FR-19 (Admin) nằm ở phân hệ **frontend-admin** riêng (cổng 5174), mở rộng phạm vi vượt quá thời lượng seminar 45 phút. FR-20 (Mobile) đã loại từ mục 1.3.

6. Thiết kế kịch bản Demo

Toàn bộ mục này đã được nhóm **chạy thật** trên EShop (Playwright 1.61.1, Node 22.22.1, Chromium headless, ngày 14/07/2026). Mọi con số, thông báo lỗi và defect nêu dưới đây đều lấy từ log thật, không phải kịch bản giả định. Chi tiết kỹ thuật đầy đủ nằm ở **User_Guide.md**; kịch bản quay theo từng phút nằm ở **Demo_Screencast_Script.md**; mã nguồn ở thư mục **eshop-e2e/**.

6.1. Mục tiêu Demo

- Thoả quy tắc pairing của brief:** thể hiện ít nhất 1 tính năng của tool truyền thống (Playwright: locator, auto-wait, trace viewer) **VÀ** ít nhất 1 tính năng của hướng AI-augmented (AI coding assistant sinh test draft) trong cùng một demo.
- Chứng minh luận điểm trung tâm của seminar:** *test xanh không đồng nghĩa phần mềm đúng*. EShop là SUT được cố ý cài lỗi (README của SUT nói rõ), nên demo không nhằm làm test xanh, mà nhằm cho thấy công cụ — và AI đặt lên trên nó — có thể **đánh lừa** chính người dùng nó theo những cách nào.
- Thiết lập một nguyên tắc làm việc có thể đem đi dùng thật: locator bám DOM thật — assertion bám đặc tả.** Trộn lẫn hai thứ này thì bộ test sẽ hợp thức hoá đúng những con bug cần tìm.

6.2. Chuẩn bị môi trường

Hạng mục	Cấu hình thật khi chạy
Runtime	Node.js 22.22.1, npm 9
Traditional tool	@playwright/test 1.61.1 + Chromium (headless)
AI direction	AI coding assistant trong IDE (Copilot Student / Claude Code / Codex — theo mục 4.6)
IDE	VS Code
SUT — Backend	cd eshop-sut/backend && npm install && node database.js && node server.js → cổng 3000
SUT — Frontend Web	cd eshop-sut/frontend-web && npm install && npm run dev → cổng 5173
Test project	eshop-e2e/ — npm i -D @playwright/test + npx playwright install chromium
Tài khoản seed	test@eshop.com / Test1234! (user), admin@eshop.com / Admin123! (admin)

Hai vấn đề môi trường nhóm đã thực sự gặp (ghi lại vì nhóm khác nhiều khả năng gặp lại):

- **Cổng 3000 bị chiếm.** Máy demo đang chạy một ứng dụng khác giữ cổng 3000 nên backend EShop không bind được. SUT hardcoded `http://localhost:3000` ở **13 vị trí** trong `frontend-web/src`, nên phải sửa cả hai phía mới đổi được cổng: `const PORT = process.env.PORT || 3000` ở backend, và thay URL tương ứng ở frontend, rồi chạy test với `API_URL=http://localhost:3001`.
- `npx playwright install --with-deps` **thất bại trên Linux** với lỗi `sudo: A terminal is required to authenticate` — cờ này cần quyền root. Cách xử lý: bỏ cờ, chạy `npx playwright install chromium`.

6.3. Các bước Demo — Traditional Tool (Playwright)

Điểm nhấn: **locator, auto-wait, và trace viewer** — ba tính năng lõi của công cụ truyền thống.

Bước 1 — Dò DOM thật trước khi viết locator (`tests/probe.spec.ts`). Đây là bước mà phần lớn tutorial bỏ qua, và cũng là bước quyết định thành bại trên một SUT có lỗi. Kết quả thật trên trang `/login` của EShop:

```
số thẻ h1      : 0
getByLabel(Username): 0      <-- KHÔNG dùng được!
số textbox    : 2
các nút       : [ 'Sign In' ]
```

Bước 2 — Đọc kết quả và rút ra chiến lược locator. `getByLabel('Username')` khớp **0 phần tử** dù mắt thường đọc rõ chữ "Username" trên màn hình: thẻ `<label>` trong `Login.jsx` không có `for/id` và không bọc input, nên không phải nhấn hợp lệ theo chuẩn accessibility (mục 2.4). SUT cũng **không có bất kỳ `data-testid` nào** (grep toàn bộ `src/`: 0 kết quả), và cả hai ô nhập đều là `type="text"` nên không phân biệt được bằng type. Hệ quả: buộc phải neo locator theo **vị trí trong form** — một locator **yếu**, và chính sự yếu đó là *bằng chứng của defect* (vi phạm FR-22), không phải một lựa chọn thiết kế.

Bước 3 — Đóng gói vào Page Object (`pages/LoginPage.ts`) để cô lập điểm yếu: khi SUT được sửa (gắn `for/id` hoặc thêm `data-testid`), chỉ cần sửa 2 dòng khai báo, không spec nào phải đụng tới — minh họa trực tiếp luận điểm maintainability ở mục 2.4.

Bước 4 — Chạy test đầu tiên (TC-L1: đăng nhập hợp lệ) → 1 passed trong 747ms, và trong toàn bộ codebase **không có một lệnh `sleep` nào**. Độ ổn định này đến từ `auto-wait` + `web-first assertion`, không phải may mắn.

Lưu ý về **test oracle**: chỉ assert `toHaveURL('/')` là oracle **yếu** — trang chủ EShop vẫn hiển thị bình thường ngay cả khi chưa đăng nhập, nên test sẽ xanh dù login hỏng. Phải khẳng định thêm một dấu hiệu **chỉ tồn tại khi đã đăng nhập** (nút "Thoát" trên header).

Bước 5 — Evidence khi fail: mở trace viewer. Chạy TC-C2 (bấm "Thêm vào giỏ hàng" đúng 1 lần ở trang chi tiết sản phẩm) → **FAIL**: `Expected: 1, Received: 0`. Mở `npx playwright show-trace ...` để tua lại từng action kèm DOM snapshot trước/sau, network và console. Trace chỉ ra: cú click **đã vào**, nhưng giỏ hàng vẫn rỗng. Đối chiếu source `ProductDetail.jsx` thì rõ nguyên nhân — **cú click đầu tiên bị nuốt có chủ đích**:

```
if (clickCount === 0) {
  setClickCount(1);
  return; // Không làm gì cả ở lần đầu tiên
}
```

Đây là **thời điểm dạy học quan trọng nhất của phần traditional**: phản xạ tự nhiên khi thấy test đỏ là "chắc do timing" rồi cho test bấm thêm lần nữa cho chắc. Làm vậy test xanh ngay — và **con bug UX biến mất vĩnh viễn khỏi báo cáo**, trong khi người dùng thật vẫn phải bấm 2 lần mỗi ngày.

6.4. Các bước Demo — AI-Augmented Direction (AI coding assistant)

Đỉnh chính so với bản nháp trước của mục này: kịch bản cũ dự kiến demo bước "Healer / self-healing". Bước đó **không áp dụng được** cho hướng AI đã chốt ở mục 4.6 — self-healing là tính năng của nền tảng SaaS (Mabl/Testim), còn AI coding assistant chỉ *sinh code*, không có cơ chế tự chữa locator lúc runtime. Mabl/Testim vẫn được giữ làm **đối chứng lý thuyết** ở mục 4 và 8.2, nhưng không nằm trong demo chạy thật.

Bước 1 — Sinh test draft từ scenario. Đưa cho AI assistant đúng một prompt, gõ trực tiếp trên màn hình khi demo:

```
Viết test Playwright cho FR-02 của EShop: người dùng đăng nhập bằng
test@eshop.com / Test1234!, sai mật khẩu 3 lần thì tài khoản bị khoá 30 giây.
```

Bước 2 — Đọc code AI sinh ra. Nó trông chuẩn mực đến mức không ai muốn nghi ngờ:

```
await expect(page.getByRole('heading', { name: 'Đăng nhập' })).toBeVisible();
await page.getByLabel('Email').fill('test@eshop.com');
await page.getByLabel('Mật khẩu').fill('Test1234!');
await page.getByRole('button', { name: 'Đăng nhập' }).click();
```

Bước 3 — Chạy thẳng, không sửa. Nhóm giữ nguyên file này (`tests/ai-draft/login-ai-draft.spec.ts`) làm **tang chứng**. Kết quả: **cả 2 test đều FAIL**, với hai kiểu lỗi khác nhau của cùng một nguyên nhân gốc:

```
# Test 1 – AI đoán sai tiêu đề trang (trang login thật lại ghi "Đăng Ký")
Error: expect(locator).toBeVisible() failed
Call log: - waiting for getByRole('heading', { name: 'Đăng nhập' })

# Test 2 – AI đoán sai nhãn ô nhập (nhãn thật là "Username", lại còn không gắn for/id)
Error: locator.fill: Test timeout of 30000ms exceeded.
Call log: - waiting for getByLabel('Email')
```

Bước 4 — Audit: vì sao AI sai? AI viết test từ **đặc tả**, chưa từng nhìn vào DOM. Nó mặc định thế giới đúng như tài liệu mô tả — mà EShop chính là hệ thống *không* như vậy. Bảng đối chiếu bản AI vs bản nhóm audit:

Tiêu chí	AI draft (chưa audit)	Bản nhóm audit
Locator ô nhập	<code>getByLabel('Email')</code> — khớp 0 phần tử	<code>form.getByRole('textbox').first()</code> — bấm DOM thật
Heading	Giả định "Đăng nhập"	Khẳng định defect: DOM đang ghi "Đăng Ký"
Oracle login	Chỉ <code>toHaveURL('/')</code> — yếu (trang chủ luôn hiện)	Thêm <code>getByRole('button', {name:'Thoát'})</code>
Oracle logout	Chờ chữ "khóa" trên UI — UI không bao giờ hiện	Kiểm ở tầng API: <code>status()</code> 401 / 403 / 200
Số điểm phải sửa	—	4/4

Bước 5 — Chỉ ra cái bẫy chết người. Phản xạ tự nhiên là dán lỗi trả lại cho AI và bảo "sửa đi". AI sẽ vui vẻ *sửa* — đổi heading mong đợi thành `'Đăng Ký'`, đổi tên nút thành `'Sign In'`. Test lập tức xanh, báo cáo đẹp — và bộ test vừa **chính thức công nhận toàn bộ đống bug đó là hành vi đúng**. Quy tắc rút ra: khi test đỏ, phải phân định *"DOM khác đặc tả"* (→ **defect của SUT**, ghi bug report) hay *"test viết sai"* (→ sửa test). **Chỉ sửa locator, tuyệt đối không nói lòng assertion.**

Kết luận của phần AI: AI tăng tốc việc gõ code, nhưng **AI không phải test oracle**.

6.5. Đo lường Flakiness

Phương pháp. Dùng cờ `--repeat-each=10` của Playwright, chạy trên Chromium local, so sánh lần chạy đầu với lần thứ 10.

Kết quả thật:

Test	Kết quả 10 lần	Flake rate
TC-C1 — thêm sản phẩm từ trang chủ vào giỏ	10/10 pass (3.5s)	0%
TC-K1 — đặt hàng thành công	10/10 pass (4.9s)	0%

Phân tích một ca "flaky" cụ thể — và đây là phát hiện đáng giá nhất của nhóm. Trong quá trình xây dựng, nhóm gặp một test cứ **fail lúc xanh lúc đỏ** một cách khó hiểu ở luồng Add-to-Cart. Điều tra bằng trace viewer cho thấy **nguyên nhân không phải flakiness**, mà là **lỗi của chính test**: nhóm dùng `page.goto('/cart')` để sang giỏ hàng, trong khi giỏ hàng của EShop nằm trong React state (`CartContext` dùng `useState`, **không** persist xuống `localStorage`). Mọi hard navigation (`goto`, F5) đều **reset giỏ về rỗng**. Test đã "phát hiện" một con bug **không hề tồn tại**.

Cách khắc phục: điều hướng bằng **click vào link trên navbar** (SPA routing giữ nguyên state) thay vì `goto`:

```
async openFromNavbar() {
  await this.page.getByRole('link', { name: 'Giỏ hàng' }).click();
  await this.page.waitForURL('**/cart');
}
```

Bài học: khi test SPA fail, câu hỏi đầu tiên phải là *"mình có vô tình reload trang không?"* — **trước** khi kết luận SUT có bug. Đây là ví dụ sống cho luận điểm ở mục 2.4: không phải test đỏ nào cũng là bug của sản phẩm, và không phải test xanh

nào cũng là bằng chứng sản phẩm đúng.

6.6. Kết quả mong đợi

Kết quả thực tế của lần chạy đầy đủ (`npx playwright test`, 40.2 giây):

23 test — 8 passed — 15 failed.

Con số 15 test đỏ **không phải thất bại của bộ test** — mà là **thành công của nó**. Mỗi test đỏ truy ngược về một defect thật của SUT:

Test	Defect phát hiện	Vi phạm
TC-L3	Ô mật khẩu là <code>type="text"</code> → mật khẩu hiện rõ nguyên văn trên màn hình	FR-22, SEC
TC-L4	Mỗi lần sai bộ đếm tăng 2 (<code>login_attempts + 2</code>) → khoá ngay sau 2 lần sai thay vì 3	FR-02
TC-L5	Khoá 180 giây thay vì 30 giây	FR-02
TC-C2	Nút "Thêm vào giỏ hàng" ở trang chi tiết phải bấm 2 lần mới ăn	FR-06
TC-C3	Thêm cùng sản phẩm 2 lần → tạo 2 dòng thay vì gộp số lượng	FR-07
TC-C4	Navbar "Giỏ hàng" không có badge số lượng	FR-23
TC-C5	Nhấn tổng tiền ghi "Tổng tạm tính" thay vì "Tổng cộng"	FR-07
TC-C6	Nút "Xóa" không có dialog xác nhận — xóa thẳng tay	FR-07, FR-24
TC-K2	Sau thanh toán thành công, giỏ hàng không được xoá	FR-08
TC-K3	Tổng tiền thanh toán là <code><input type="number"></code> — người dùng sửa được	FR-08
TC-K4	Backend chấp nhận <code>total_amount</code> do client gửi → mua iPhone 30 triệu với giá 1.000 đ	FR-08, SEC
AI draft (2 test)	Không phải bug SUT — bug của test do AI sinh (xem 6.4)	—

Hai kết quả đắt giá nhất, sẽ được demo trực tiếp trước lớp:

(a) Lỗ hổng bảo mật ở Checkout (TC-K4). Sửa ô tổng tiền từ 30.000.000 đ xuống **1.000 đ** ngay trên trình duyệt rồi bấm thanh toán → UI hiện to đùng **"Thanh toán thành công!"**. Nếu oracle chỉ nhìn UI — đúng y như bản AI draft làm — test sẽ **PASS**. Nhóm đặt oracle ở **tầng dữ liệu**, hỏi thẳng API đơn hàng vừa tạo:

Error: backend không được nhận tổng tiền do client gửi
Expected: 30000000
Received: 1000

Backend **thật sự** đã ghi đơn 30 triệu với giá 1.000 đ. Câu hỏi của người kiểm thử không bao giờ được là *"màn hình có hiện chữ thành công không?"*, mà phải là **"bằng chứng nào chứng minh nghiệp vụ đã đúng?"**

(b) Test xanh giả — auto-wait KHÔNG bảo vệ assertion phủ định. Ai cũng tin "Playwright có auto-wait nên khỏi lo timing". Nhóm chứng minh điều đó **sai** với `toHaveCount(0)` / `not.toBeVisible()`: chúng được thoả mãn **ngay ở lần poll đầu tiên**, lúc SPA còn chưa kịp render trang mới — không có gì để chờ, nên "đúng" tức thì. Thí nghiệm đối chứng

(tests/false-pass-demo.spec.ts) — cùng một yêu cầu FR-08, cùng một SUT, chỉ khác đúng một dòng neo trạng thái:

```
// ❌ XANH GIẢ – assert ngay sau click, lúc /checkout chưa render
await expect(page.locator('input[type="number"]')).toHaveCount(0); // ✓ passed
(949ms)

// ✅ ĐỎ ĐÚNG – neo trạng thái trước rồi mới assert
await expect(page.getByRole('button', { name: 'Xác Nhận Thanh Toán' })).toBeVisible(); // NEO
await expect(page.locator('input[type="number"]')).toHaveCount(0); // ❌
failed: Expected 0, Received 1
```

Kết quả chạy thật: **1 passed, 1 failed**. Bản "xanh" **bỏ lọt** defect FR-08 trong im lặng. Quy tắc: trước mọi assertion phải định, phải **neo** vào một phần tử *chắc chắn tồn tại* của trạng thái đích — nếu không, bạn chỉ đang chứng minh "trang cũ không có thứ đó", một sự thật vô nghĩa.

6.7. Rủi ro & phương án dự phòng

Rủi ro	Xác suất	Phương án dự phòng
Cổng 3000/5173 bị chiếm trên máy demo	Cao — đã xảy ra thật với nhóm	Kiểm tra <code>ss -ltnp grep :3000</code> trước buổi seminar. Nếu bận: chạy <code>PORT=3001 node server.js</code> + sửa URL ở frontend + chạy test với <code>API_URL=http://localhost:3001</code> . Chuẩn bị sẵn nhánh code đã cấu hình 3001.
AI assistant hết quota / mạng chết đúng lúc demo (Scene AI)	Trung bình	Bản AI draft đã được lưu sẵn trong <code>tests/ai-draft/</code> — vẫn chạy được offline và cho ra đúng 2 lỗi cần trình bày. Không phụ thuộc việc AI phải sinh code live.
Mất mạng hoàn toàn	Thấp	Toàn bộ SUT + test chạy local , không cần Internet sau khi đã <code>npm install</code> và tải Chromium. Đây chính là lý do nhóm chọn Playwright thay vì Mabl/Testim (SaaS, bắt buộc cloud) — xem mục 4.6.
Test 31 giây (AI draft lockout) làm cháy thời lượng demo	Trung bình	Test này timeout 30s theo thiết kế. Khi demo, chạy nó ở terminal nền từ trước , hoặc tua nhanh phần chờ — nhưng không được cắt ghép giả kết quả (brief chấm phạt "pre-recording the live demo").
Dữ liệu SUT bị bẩn sau nhiều lần chạy (tài khoản bị khoá, giỏ hàng rác)	Cao	Fixture <code>freshUser</code> tự tạo user mới qua API cho từng test (<code>e2e-<timestamp>@eshop.test</code>), nên test lockout không làm khoá tài khoản của test khác. Muốn reset sạch: chạy lại <code>node database.js</code> .
Máy demo hỏng / sự cố phòng máy	Thấp	<code>Demo_Screencast.mp4</code> đã quay sẵn đóng vai trò bản dự phòng (brief yêu cầu "have a backup recording ready in case the network dies").

7. Hoạt động thực hành cho khán giả (In-class Activity)

7.1. Mục tiêu hoạt động

- Giúp khán giả **tự trải nghiệm** sự khác biệt giữa locator viết tay và locator do AI sinh ra, thay vì chỉ nghe lý thuyết.

- Minh hoạ trực tiếp các khái niệm lý thuyết đã trình bày: Locator strategy (mục 2.4), Brittle Locator/Flaky Test (bảng rủi ro mục 2.4), vai trò và rủi ro của AI coding assistant (mục 2.5).
- Tạo ra một **rubric "good locator"** được cả lớp đồng thuận ngay tại chỗ — sản phẩm cụ thể khán giả mang về, không chỉ là hoạt động giải trí.
- Đáp ứng yêu cầu bắt buộc của brief T02: hoạt động phải **tái lập được trong ≤ 25 phút bởi một nhóm khác mà không cần hỗ trợ của nhóm thuyết trình** (mục 9 — Topic-Specific Grading Notes).

7.2. Tên & mô tả hoạt động: "Locator Brawl — Hand-crafted vs AI-suggested"

Hai đội khán giả cùng viết một test Add-to-Cart trên EShop theo hai cách khác nhau, sau đó so sánh kết quả trực tiếp:

- **Đội A (viết tay / hand-crafted locator):** viết test bằng Playwright, **được phép dùng AI hỗ trợ cú pháp** (cách viết `test()`, `expect()`, cấu trúc file...) vì phần lớn khán giả lần đầu tiếp cận Playwright trong thời gian ngắn — nhưng **tự mình chọn locator**, không để AI gợi ý locator, theo best practice đã học ở mục 2.4 (ưu tiên role/label/text/test-id).
- **Đội B (AI-suggested locator):** mô tả scenario bằng ngôn ngữ tự nhiên cho một AI coding assistant bất kỳ mà đội có sẵn (Copilot/Claude/Codex hoặc tương đương) để **AI tự quyết định cả locator lẫn cách viết test**, đội chỉ chỉnh sửa tối thiểu để chạy được — không tự ý đổi locator AI đã chọn. *Cố ý không giới hạn cứng vào 1 tool AI cụ thể* — vì đội khán giả nhiều khả năng không có cùng tài khoản trả phí như nhóm thuyết trình (Claude Max, Codex Pro), nên bắt buộc phải linh hoạt để đảm bảo tái lập được (xem mục 7.6).
- **Biến số so sánh duy nhất là "ai chọn locator"** (người hay AI), không phải "ai gõ code" — cả hai đội đều được dùng AI hỗ trợ viết code, chỉ khác ở việc ai ra quyết định locator.
- Cả hai đội dùng chung một flow, một SUT (EShop Add-to-Cart) và cùng dữ liệu test, đảm bảo so sánh công bằng.

7.3. Timeline chi tiết (tối đa 25 phút)

Trước 0:00 (không tính vào 25 phút): mỗi đội tự host một bản EShop riêng trên máy mình (không dùng chung 1 instance với đội khác — tránh 2 đội cùng ghi vào 1 giỏ hàng làm sai lệch kết quả, vi phạm Test Isolation ở mục 2.4). Chi tiết lệnh cụ thể ở [Activity_Worksheet.md](#) mục A.

Thời gian	Nội dung	Ghi chú thực hiện
0:00–0:03	Facilitator demo flow mẫu trên EShop (login → add to cart → assert badge số lượng)	Chạy trực tiếp, không dùng slide, để khán giả thấy rõ hành vi mong đợi trước khi tự viết
0:03–0:10	Đội A viết test tay bằng Playwright; Đội B mô tả scenario cho AI, review code AI sinh ra	Cả hai đội chỉ được truy cập docs chính thức (Playwright/AI tool), không hỏi nhóm thuyết trình
0:10–0:15	Mỗi đội chạy test 3 lần với mạng đã giả lập chậm ngay trong code test, dùng snippet dựng sẵn ở mục 7.4	Mô phỏng điều kiện thực tế gây flaky một cách nhất quán giữa các đội — liên hệ trực tiếp bảng rủi ro ở mục 2.4
0:15–0:20	Hai đội trình bày kết quả: loại locator đã dùng, số lần fail/pass, lý do fail nếu có	Facilitator ghi nhận lên bảng để cả lớp cùng thấy
0:20–0:25	Cả lớp thống nhất rubric "good locator" dựa trên kết quả vừa quan sát (dùng khung ở mục 7.5)	Facilitator tổng hợp ý kiến, chốt lại 3-4 tiêu chí đồng thuận

7.4. Vật liệu / Worksheet cần chuẩn bị

Nộp riêng theo Stage S5 dưới dạng file [Activity_Worksheet.md](#) (đúng checklist deliverable "worksheet + answer key" — không lặp lại toàn bộ nội dung trong báo cáo này, giống cách [Tool_Survey_Proposal.md](#)/[User_Guide.md](#)/[\[AI-02/03/04\]](#) đều là file riêng, xem mục 10.4).

Tóm tắt nội dung file đó:

- **Phần chuẩn bị trước hoạt động** (không tính vào 25 phút): mỗi đội tự host một bản EShop riêng trên máy mình bằng lệnh clone + `npm install + node server.js/npm run dev` thật từ repo (đã xác minh có sẵn `setup_guide.md` và script `run_servers.sh` hỗ trợ khởi động nhanh) — tự host để tránh nhiều đội cùng ghi vào chung 1 giỏ hàng, vì phạm Test Isolation (mục 2.4).
- **Phần phát cho khán giả**: lệnh setup Playwright nhanh, scenario & test data cụ thể (flow Add-to-Cart, tài khoản test có sẵn trong dữ liệu mẫu của repo, oracle là số trên badge giỏ hàng), bảng luật chơi riêng cho Đội A/Đội B, bảng trống để đội tự ghi nhận kết quả 3 lần chạy, snippet throttle mạng (mục 7.3), và rubric "good locator" (mục 7.5).
- **Phần Answer Key** (chỉ dành cho facilitator, không phát khán giả): kết quả dự kiến của từng đội, cách xử lý nếu kết quả lệch kỳ vọng (cả 2 đội đều pass, hoặc Đội B không kịp xong), và cách dẫn dắt thảo luận 5 phút cuối dựa trên dữ liệu thật thay vì chốt kết luận trước.

7.5. Tiêu chí đánh giá / Rubric "good locator"

Khung rubric facilitator dùng để dẫn dắt thảo luận ở mốc 0:20–0:25 — không áp đặt sẵn kết luận mà để lớp tự đối chiếu với kết quả thực tế vừa chạy:

Tiêu chí	Yếu (1 điểm)	Trung bình (2 điểm)	Tốt (3 điểm)
Loại locator	XPath/CSS theo vị trí DOM sâu (VD: <code>div > div:nth-child(3) > button</code>)	CSS class/id dùng chung với style giao diện	Role/Label/Text/Test-id (VD: <code>getByRole, data-testid</code>)
Độ ổn định qua 3 lần chạy	Fail ít nhất 1 lần	Pass nhưng cần retry/không ổn định	Pass ổn định cả 3 lần
Độc lập với style/layout	Gắn liền class dùng cho CSS, dễ vỡ khi đổi giao diện	Một phần độc lập	Hoàn toàn tách biệt khỏi style (thuộc tính test riêng)
Dễ đọc / thể hiện đúng ý định	Khó đoán locator đang nhắm tới phần tử nào	Đoán được nhưng dài dòng	Rõ ràng, đúng ý định của test

Sau khi thảo luận, đối chiếu với 3 kết luận đã có sẵn trong brief T02 (audience takeaways) để cả lớp kiểm chứng lại bằng chính kết quả mình vừa chạy, thay vì chỉ đọc lý thuyết suông:

- AI locator có xu hướng chọn XPath/CSS ngắn nhưng dễ gãy khi DOM đổi nhỏ.
- Thuộc tính `data-test-id` vẫn thắng về khả năng bảo trì lâu dài.
- Self-healing giảm nhiễu nhưng có thể che giấu lỗi thật — nên đi kèm visual diff/evidence (liên hệ mục 2.4, 8.2).

7.6. Khả năng tái lập bởi nhóm khác

- **Không phụ thuộc tài khoản trả phí của nhóm thuyết trình**: Đội B được chọn bất kỳ AI coding assistant nào đội đang có sẵn (kể cả bản miễn phí), không bắt buộc phải có Claude Max/Codex Pro như nhóm thuyết trình.
- **Không phụ thuộc instance EShop hay mạng do nhóm thuyết trình host**: mỗi đội tự host EShop cục bộ trên máy mình (repo công khai, có sẵn script khởi động nhanh) — không cần kết nối tới máy/mạng của nhóm thuyết trình, tránh rủi ro nghẽn mạng hoặc mất kết nối giữa chừng.
- **Không phụ thuộc cài đặt phức tạp**: chỉ cần Playwright (`npm init playwright@latest`) và trình duyệt có DevTools — cả hai đều miễn phí, cài trong vài phút; phần tự host EShop được làm trước, không tính vào 25 phút hoạt động.
- **Không cần kiến thức nền sâu**: flow Add-to-Cart đơn giản, chỉ cần đọc qua mục 2.4 (Locator) là đủ để tham gia.
- Xác nhận hoạt động có thể thực hiện độc lập trong ≤ 25 phút, không cần nhiều hỗ trợ từ nhóm thuyết trình — đúng yêu cầu bắt buộc "Activity worksheet MUST be reproducible by a peer team in ≤ 25 minutes without your help" (T02 brief, mục 9).

8. Chi phí bảo trì & Failure Modes

8.1. So sánh chi phí bảo trì: Test AI-generated vs Test viết tay

Quan sát ngắn hạn (trong phạm vi chuẩn bị seminar)

Tiêu chí	Test viết tay (Playwright)	Test AI-generated (Copilot/Claude/Codex)
Tốc độ viết lần đầu	Chậm hơn — cần tự nhớ API, tự chọn locator	Nhanh hơn — sinh draft gần như ngay lập tức
Số lần cần chỉnh sửa trước khi pass đúng oracle	Ít hơn nếu người viết đã quen Playwright	Thường cần ít nhất 1 vòng sửa lại locator/assertion
Locator ban đầu	Chủ động chọn role/label/test-id (mục 2.4)	Cần review kỹ — dễ ra CSS/XPath bám cấu trúc DOM nếu prompt không hướng dẫn rõ

Suy luận về chi phí bảo trì dài hạn (4–6 tuần) — dựa trên lý thuyết đã kiểm chứng

- **Test viết tay:** chi phí sửa tập trung vào các thời điểm UI thay đổi thật sự ảnh hưởng locator đã chọn — thường ít nếu dùng role/label/test-id (mục 2.4) — nhưng đòi hỏi kỷ luật và thời gian đầu tư ban đầu cao hơn.
- **Test AI-generated:** chi phí ban đầu thấp nhưng rủi ro tích lũy theo thời gian nếu không review kỹ ngay từ đầu — đúng cảnh báo đã nêu ở mục 2.5 ("locator mong manh, assertion thiếu nghiệp vụ nếu không review") và mục 4.5 Nhược điểm ("code sinh ra cần review kỹ"). Nếu để lọt locator mong manh từ đầu, chi phí sửa dồn vào các đợt UI thay đổi sau — thường tốn công hơn sửa một lần dứt điểm ngay từ đầu.
- **Điểm giao nhau quan trọng:** chi phí bảo trì thực sự phụ thuộc vào **chất lượng review ban đầu**, không phải việc có dùng AI hay không. AI không làm tăng/giảm chi phí bảo trì một cách tất định — nó khuếch đại rủi ro nếu review lỏng, và khuếch đại tốc độ nếu review chặt.

Khuyến nghị thực hành: dùng AI để tăng tốc draft ban đầu, nhưng bắt buộc review locator theo checklist ở mục 2.4 trước khi merge — biến chi phí review ban đầu thành khoản đầu tư giảm chi phí bảo trì dài hạn (áp dụng trực tiếp ở mục 9).

8.2. Failure Modes (bắt buộc ≥ 3 tình huống công cụ có thể gây hiểu nhầm)

Không dùng ví dụ giả định — cả 3 tình huống dưới đây đều là phát hiện đã kiểm chứng trực tiếp từ tài liệu chính thức ở chương 4, không phải suy đoán:

1. **Self-healing tự động áp dụng, không cần phê duyệt con người (Testim, mục 4.3):** Auto Improve của Testim tự thay locator ngay khi điểm giảm dưới 70%, không chờ xác nhận. Nếu UI đổi do **bug thật** (không phải thay đổi cố ý), Testim vẫn có thể "chữa" locator để tiếp tục pass — che giấu chính bug cần phát hiện (False Pass, mục 2.4). Người dùng tin "test pass" trong khi ứng dụng đã có lỗi.
2. **Visual change chỉ là cảnh báo, không chặn fail (Mabl, mục 4.2):** nếu không chủ động xem lại cảnh báo, một lỗi giao diện thật (nút bị lệch, mất phần tử quan trọng) vẫn "pass" vì assertion chức năng không kiểm tra được sai khác hình ảnh — dễ khiến người dùng tin hệ thống ổn trong khi giao diện đã hỏng.
3. **Thông điệp "governance" không nhất quán với hành vi thực tế (Katalon, mục 4.4):** Katalon quảng bá True Platform "auditable" và "con người phê duyệt", nhưng cùng tài liệu công bố lại ghi self-healing hoạt động "không cần con người can thiệp". Người dùng tin vào thông điệp marketing tổng quát có thể đánh giá sai mức độ kiểm soát thực tế của từng tính năng cụ thể.
4. **Assertion AI-generated pass nhưng không đúng oracle nghiệp vụ (Copilot/Claude/Codex, mục 4.5):** nếu prompt mô tả không đủ rõ, AI có thể sinh assertion chỉ kiểm tra "phần tử hiển thị" hoặc "click thành công" thay vì

đúng oracle nghiệp vụ (ví dụ số trên badge giỏ hàng tăng đúng 1 — nguyên tắc đã nhấn mạnh ở [Activity_Worksheet.md](#) mục C) — test pass nhưng không thực sự kiểm chứng đúng hành vi mong đợi.

9. Khuyến nghị cho đội phát triển EShop (Recommendation Memo)

To: Đội phát triển EShop

From: Nhóm seminar T02 — Web Automation Testing

Re: Đề xuất chiến lược Web Automation Testing

1. Công cụ chính: dùng **Playwright (TypeScript)** cho toàn bộ E2E test trên Frontend Web — lý do chi tiết đã trình bày ở mục 3.8 (auto-wait giảm flaky từ thiết kế, Trace Viewer cho evidence khi fail, không phụ thuộc dịch vụ cloud trả phí để chạy song song). Không cần đầu tư thêm Selenium/Cypress/Puppeteer trừ khi có nhu cầu đặc thù (ví dụ Selenium nếu cần tích hợp hệ thống cũ đã có sẵn).

2. Vai trò của AI: dùng GitHub Copilot/Claude Code/Codex (tuỳ công cụ sẵn có của từng dev) để tăng tốc viết draft test ban đầu — nhưng **bắt buộc review locator/assertion theo checklist mục 2.4 trước khi merge**, không merge trực tiếp code AI sinh ra. Đây là điều kiện tiên quyết để chi phí bảo trì dài hạn thấp hơn chi phí ban đầu tiết kiệm được (mục 8.1).

3. Không nên đầu tư vào nền tảng AI-native SaaS đóng (Mabl/Testim/Katalon) ở giai đoạn hiện tại của EShop: cả ba đều có chi phí license đáng kể khi cần chạy CI/CD (mục 4.2-4.4), trong khi lợi ích self-healing có thể đạt được một phần bằng kỹ thuật viết locator tốt (role/label/test-id) với chi phí thấp hơn nhiều. Chỉ nên cân nhắc lại nếu quy mô test suite tăng đủ lớn để chi phí bảo trì thủ công vượt chi phí license.

4. Bắt buộc theo dõi 2 chỉ số: tỉ lệ flaky test (đo theo cách ở mục 6.5) và số lần phải sửa test AI-generated sau mỗi đợt release UI — theo dõi thật trong 4-6 tuần đầu triển khai (điều seminar không có đủ thời gian làm) để xác nhận lại suy luận ở mục 8.1 bằng dữ liệu thật của chính EShop, không dừng ở suy luận lý thuyết.

5. Ưu tiên luồng nghiệp vụ giá trị cao trước: áp dụng đúng tiêu chí chọn test case đã nêu ở mục 2.3 — không cố phủ toàn bộ giao diện ngay từ đầu.

Tài liệu tham khảo

- ISTQB Glossary — Test Case: https://glossary.istqb.org/en_US/term/test-case
- ISTQB Glossary — Test Suite: https://glossary.istqb.org/en_US/term/test-suite
- ISTQB Glossary — Regression Testing: https://glossary.istqb.org/en_US/term/regression-testing
- ISTQB Glossary — Test Oracle: https://glossary.istqb.org/en_US/term/test-oracle-4-2
- James Bach — Test Automation Snake Oil (1999): <https://www.satisfice.com/download/test-automation-snake-oil>
- Martin Fowler — TestPyramid: <https://martinfowler.com/bliki/TestPyramid.html>
- Martin Fowler — PageObject: <https://martinfowler.com/bliki/PageObject.html>
- MDN Web Docs — Document Object Model (Introduction): https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction
- Google Testing Blog — Flaky Tests at Google and How We Mitigate Them (2016): <https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html>
- mabl — Self-Healing Test Automation for Autonomous QA: <https://www.mabl.com/blog/self-healing-test-automation-autonomous-qa>
- Playwright — Getting Started (ngôn ngữ hỗ trợ, cài đặt, codegen): <https://playwright.dev/docs/intro>
- Playwright — Browsers (kiến trúc engine, browser context): <https://playwright.dev/docs/browsers>

- Playwright — Locators: <https://playwright.dev/docs/locators>
- Playwright — Auto-waiting/Actionability: <https://playwright.dev/docs/actionability>
- Playwright — Assertions: <https://playwright.dev/docs/test-assertions>
- Playwright — Trace Viewer: <https://playwright.dev/docs/trace-viewer-intro>
- Playwright — Parallelism & Sharding: <https://playwright.dev/docs/test-parallel>
- Playwright — Continuous Integration: <https://playwright.dev/docs/ci-intro>
- Playwright — Test Agents (Planner/Generator/Healer): <https://playwright.dev/docs/test-agents>
- Selenium — Documentation Overview (ngôn ngữ hỗ trợ, thành phần dự án): <https://www.selenium.dev/documentation/>
- Selenium — WebDriver: <https://www.selenium.dev/documentation/webdriver/>
- Selenium — Grid: <https://www.selenium.dev/documentation/grid/>
- Selenium — Locators: <https://www.selenium.dev/documentation/webdriver/elements/locators/>
- Selenium — Waits: <https://www.selenium.dev/documentation/webdriver/waits/>
- Selenium — Page Object Models: https://www.selenium.dev/documentation/test_practices/encouraged/page_object_models/
- Cypress — Why Cypress (giới thiệu, kiến trúc trong-trình-duyệt): <https://docs.cypress.io/app/get-started/why-cypress>
- Cypress — Best Practices (khuyến nghị `data-*` locator, test isolation): <https://docs.cypress.io/app/core-concepts/best-practices>
- Cypress — Retry-ability: <https://docs.cypress.io/app/core-concepts/retry-ability>
- Cypress — Screenshots and Videos: <https://docs.cypress.io/app/guides/screenshots-and-videos>
- Cypress — Continuous Integration Overview: <https://docs.cypress.io/app/continuous-integration/overview>
- Cypress — AI Skills (cypress-author/explain/docs): <https://docs.cypress.io/app/tooling/ai-skills>
- Cypress — Cloud MCP: <https://www.cypress.io/blog/cloud-mcp-give-your-ai-assistant-access-to-your-test-runs>
- Selenium MCP (cộng đồng, không chính thức) — ví dụ tham khảo: <https://github.com/angiejones/mcp-selenium>
- Puppeteer — What is Puppeteer (giới thiệu, cài đặt, test runner): <https://pptr.dev/guides/what-is-puppeteer>
- Puppeteer — Page.waitForSelector(): <https://pptr.dev/api/puppeteer.page.waitForselector>
- Puppeteer MCP Server (tham chiếu, cộng đồng MCP gốc): <https://github.com/merajmehrabi/puppeteer-mcp-server>
- mabl — Pricing (xác nhận chỉ có trial 14 ngày, không có free tier): <https://www.mabl.com/pricing>
- mabl — How auto-heal works: <https://help.mabl.com/hc/en-us/articles/19078583792404-How-auto-heal-works>
- mabl — Visual change detection overview: <https://help.mabl.com/docs/visual-testing-and-monitoring>
- mabl — CI/CD integrations: <https://help.mabl.com/hc/en-us/sections/16282691422612-CI-CD-integrations>
- mabl — Test Automation Results and Analysis (root cause analysis): <https://www.mabl.com/test-automation-results-and-analysis>
- Testim — Smart Locators (Tricentis blog): <https://www.tricentis.com/blog/testim-locator-technologies>
- Testim — Locators: Auto Improve: <https://docs.tricentis.com/testim/content/test-management/locators-auto-improve.htm>
- Testim — Pricing (xác nhận Community free plan vĩnh viễn sau trial): <https://www.testim.io/pricing/>
- Testim — CI integrations: <https://docs.tricentis.com/testim/content/integrations/integrate-testim-to-your-ci/index.htm>
- Testim — Root Cause Analysis: <https://www.testim.io/root-cause-analysis/>
- Testim — TestOps: <https://www.testim.io/testops/>
- Katalon — Launches True Platform (04/2026, AI agents, governance): <https://katalon.com/resources-center/blog/katalon-launches-true-platform>
- Katalon Docs — Jenkins integration overview: <https://docs.katalon.com/katalon-studio/integrations/cicd-integrations/jenkins-integration/jenkins-integration-overview>
- Katalon — Pricing (xác nhận Studio Forever Free, Runtime Engine trả phí): <https://katalon.com/pricing>
- GitHub Copilot — Writing tests (chế độ chat không tự chạy test): <https://docs.github.com/en/copilot/tutorials/write-tests>

- GitHub Copilot — Plans (Free/Pro/Pro+/Max/Business/Enterprise/Student): <https://docs.github.com/en/copilot/get-started/plans>
- GitHub Copilot — About coding agent (chạy test trong sandbox GitHub Actions): <https://docs.github.com/copilot/concepts/agents/coding-agent/about-coding-agent>
- Playwright MCP — Introduction: <https://playwright.dev/mcp/introduction>
- Claude — Pricing (xác nhận Claude Code cần gói Pro trở lên, không có free): <https://claude.com/pricing>
- OpenAI Codex — Pricing (xác nhận có Free tier): <https://learn.chatgpt.com/docs/pricing>